



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH

**Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique**

Licence d'éducation : Spécialité Enseignement Secondaire - Informatique

Groupe : [Groupe 1 / Groupe 2]

Semestre S6

Module : Développement Web et Mobile 2

Rapport de Projet de Fin de Module

*Conception et développement d'une application web et mobile pour la
maintenance des équipements (salles / labos)*

Réalisé par

Salma LAKHAL

Encadré par

Pr. Mohamed Lachgar

Année universitaire : 2025–2026

Déclaration

Je soussignée, **Salma LAKHAL**, étudiante à l'École Normale Supérieure de Marrakech, Université Cadi Ayyad, Département d'Informatique, déclare sur l'honneur que le présent rapport, intitulé « *Conception et développement d'une application web et mobile pour la maintenance des équipements (salles / labos)* », constitue le fruit de mon travail personnel et original.

Je certifie que :

- L'ensemble des textes, figures, tableaux, extraits de code et illustrations sont le résultat de mon propre travail, sauf indication contraire explicite.
- Toute source externe utilisée (ouvrages, articles, sites web, logiciels, etc.) a fait l'objet d'une citation et d'une référence bibliographique conforme aux normes académiques en vigueur.
- Je n'ai ni reproduit ni imité sans citation des travaux d'autrui, que ce soit partiellement ou intégralement.

Je reconnais que tout manquement à cet engagement, notamment le plagiat, la contrefaçon ou la falsification de données, constitue une infraction académique grave passible de sanctions disciplinaires conformément au règlement intérieur de l'établissement.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques et de recherche, au bénéfice des futurs étudiants et chercheurs, dans le respect des droits d'auteur et de la propriété intellectuelle.

Signature :

Fait à Marrakech, le 14 mai 2026

Salma LAKHAL

Remerciements

Je tiens à remercier vivement **Pr. Mohamed Lachgar**, professeur du module **Développement Web et Mobile 2** au département d'informatique de l'ENS Marrakech, pour son encadrement, ses conseils précieux et sa disponibilité tout au long de la réalisation de ce projet. Sa rigueur pédagogique et son accompagnement m'ont permis de mener à bien ce travail dans les meilleures conditions.

Je remercie également l'ensemble des enseignants du département d'informatique pour la qualité de leur formation.

Merci à tous pour votre soutien.

Abstract

This report presents the design and implementation of a complete maintenance management platform for equipment in classrooms and laboratories at ENS Marrakech. The main objective is to digitize the fault reporting and tracking process, replacing traditional methods (paper, emails) with a centralized, efficient, and secure solution.

Achievements and measurable results :

- **Database** : MySQL database with exactly 4 tables (users, equipments, maintenance_tickets, ticket_logs) respecting the pedagogical constraint of the project.
- **Backend API** : REST API developed with Node.js and Express, exposing 15 validated endpoints (authentication, equipment management, ticket management, statistics). Security ensured by JWT (JSON Web Token) and bcrypt for password hashing.
- **Mobile application** : Android application developed in Kotlin, with 5 main screens (login, ticket list, ticket creation, ticket detail, profile). Successfully tested on emulator and physical device (API 24+).
- **Web interface** : Administration interface developed with React and Tailwind CSS, including an interactive dashboard with graphs (Chart.js), complete equipment management (CRUD), and advanced ticket management (filters, assignment, status change).
- **Tests** : 100% of API routes tested with Postman, JWT authentication verified, role-based access control validated (user/tech/admin), application of prepared SQL queries to prevent injections.
- **Delivered components** : Functional API, Android application APK, web interface deployed locally, complete technical documentation, and user manual.

The platform allows users to report a fault (equipment selection, description, priority), technicians to update status and add comments, and administrators to supervise the entire system through a dashboard with real-time statistics. Full traceability is ensured by the `ticket_logs` table, which records all actions performed on each ticket.

This project enabled the acquisition of advanced skills in Android mobile development (Kotlin, Retrofit, Coroutines), backend development (Node.js, REST API, JWT), frontend development (React, Tailwind CSS), database design (MySQL, UML modeling), and project management (Scrum methodology, scientific writing in $\text{\LaTeX}$).

Keywords : maintenance management, Android mobile application, REST API, JWT, React, Node.js, Kotlin, tickets, action history, traceability, 4 tables.

GitHub Link : <https://github.com/salma-lakhal/maintenance-pro>

Résumé

Ce rapport présente la conception et la réalisation d'une plateforme complète de gestion de maintenance des équipements dans les salles de cours et laboratoires de l'ENS Marrakech. L'objectif principal est de digitaliser le processus de signalement et de suivi des pannes, en remplaçant les méthodes traditionnelles (papier, courriels) par une solution centralisée, efficace et sécurisée.

Réalisations et résultats mesurables :

- **Base de données** : Base de données MySQL avec exactement 4 tables (users, equipments, maintenance_tickets, ticket_logs) respectant la contrainte pédagogique du projet.
- **API Backend** : API REST développée avec Node.js et Express, exposant 15 endpoints validés (authentification, gestion des équipements, gestion des tickets, statistiques). Sécurité assurée par JWT (JSON Web Token) et bcrypt pour le hachage des mots de passe.
- **Application mobile** : Application Android développée en Kotlin, avec 5 écrans principaux (connexion, liste des tickets, création de ticket, détail du ticket, profil). Testée avec succès sur émulateur et appareil physique (API 24+).
- **Interface web** : Interface d'administration développée avec React et Tailwind CSS, comprenant un tableau de bord interactif avec graphiques (Chart.js), une gestion complète des équipements (CRUD) et une gestion avancée des tickets (filtres, assignation, changement de statut).
- **Tests et validation** : 100% des routes API testées avec Postman, authentification JWT vérifiée, contrôle d'accès basé sur les rôles validé (user/tech/admin), application de requêtes SQL préparées pour prévenir les injections.
- **Composants livrés** : API fonctionnelle, APK de l'application Android, interface web déployée localement, documentation technique complète et manuel d'utilisation.

Méthodologie de validation :

La validation de la solution a été réalisée selon une approche systématique en trois phases :

1. **Tests unitaires** : Vérification individuelle de chaque fonction du backend (contrôleurs, middlewares, modèles) à l'aide de la console Node.js et de logs détaillés.
2. **Tests d'intégration** : Validation des interactions entre les composants (mobile API, web API) via des scénarios réaliste (connexion, création de ticket, changement de statut). Utilisation de Postman pour automatiser les requêtes API et vérifier les codes HTTP (200, 401, 403, 404).
3. **Tests de non-régression** : Exécution de l'ensemble des fonctionnalités après chaque modification majeure pour garantir la stabilité du système. Les tests ont été effectués sur trois environnements différents (Windows 11, Ubuntu 22.04, macOS Ventura).

La plateforme permet aux utilisateurs de signaler une panne (sélection d'équipement, description, priorité), aux techniciens de mettre à jour le statut et d'ajouter des commentaires, et aux administrateurs de superviser l'ensemble du système via un tableau de bord avec des statistiques en

temps réel. Une traçabilité complète est assurée par la table `ticket_logs`, qui enregistre toutes les actions effectuées sur chaque ticket.

Acquis et compétences développées :

Ce projet a permis d'acquérir des compétences avancées en développement mobile Android (Kotlin, Retrofit, Coroutines), développement backend (Node.js, API REST, JWT), développement frontend (React, Tailwind CSS, graphiques), conception de bases de données (MySQL, modélisation UML), et gestion de projet (méthodologie Scrum, rédaction scientifique en $\text{\LaTeX}$).

Mots clés : gestion de maintenance, application mobile Android, API REST, JWT, React, Node.js, Kotlin, tickets, historique des actions, traçabilité, 4 tables.

Liste des abréviations

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
GSON	Google JSON Library
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
MySQL	My Structured Query Language
REST	Representational State Transfer
REST API	Representational State Transfer Application Programming Interface
SQL	Structured Query Language
UI	User Interface
UX	User Experience

Table des figures

1.1	Schéma du cycle Scrum et de ses principales étapes	5
2.1	Diagramme des cas d'utilisation de la plateforme de maintenance	9
4.1	Architecture globale de la plateforme de maintenance	14
4.2	Diagramme de classes de la base de données	15
4.3	Diagramme de séquence – Authentification	16
4.4	Diagramme de séquence – Création d'un ticket	16
4.5	Diagramme de séquence – Changement de statut	17
5.1	Écran de connexion (Android)	27
5.2	Liste des tickets (Android)	28
5.3	Création d'un ticket (Android)	29
5.4	Détail d'un ticket avec historique (Android)	30
5.5	Écran de connexion (Web)	31
5.6	Tableau de bord (Web)	31
5.7	Liste des équipements (Web) – Consultation (Lire)	31
5.8	Création d'un équipement (Web) – Créer	32
5.9	Exemple de création d'un équipement (Web)	32
5.10	État de l'équipement avant modification	33
5.11	Modification d'un équipement (Web) – Modifier	33
5.12	État de l'équipement après modification	34
5.13	Confirmation de suppression d'un équipement (Web)	34
5.14	Liste des équipements après suppression	35
5.15	Liste des tickets (Web)	35
5.16	Détail d'un ticket avec historique (Web)	36

Liste des tableaux

1.1	Planification des sprints	6
3.1	Tableau comparatif des solutions existantes	12
5.1	Principaux endpoints de l'API REST	22
5.2	Principales pages et composants de l'interface web	24
5.3	Dépendances principales du projet web	25
5.4	Principales activités et fragments de l'application mobile	26
5.5	Résultats des tests	37
A.1	Résultats des tests fonctionnels	48

Table des matières

Remerciements	ii
Abstract	iii
Résumé	iv
Liste des abréviations	vi
Table des figures	vii
Liste des tableaux	viii
Introduction Générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Présentation du projet	3
1.2.1 Sujet du projet	3
1.2.2 Intérêt du projet	4
1.3 Problématique et état de l'existant	4
1.4 Objectifs du projet	4
1.4.1 Objectifs fonctionnels	4
1.4.2 Objectifs techniques	5
1.4.3 Contraintes	5
1.5 Démarche de gestion de projet	5
1.5.1 Méthodologie	5
1.5.2 Planification du projet	5
1.6 Conclusion	6
2 Étude préliminaire et fonctionnelle	7
2.1 Introduction	7
2.2 Étude des besoins	7
2.2.1 Besoins fonctionnels	7
2.2.2 Besoins non fonctionnels	7
2.3 Identification des acteurs	8
2.4 Cas d'utilisation et diagrammes	8
2.4.1 Diagramme des cas d'utilisation	8
2.4.2 Description textuelle des cas d'utilisation	9
2.5 Processus métier	10
2.6 Conclusion	10

3	État de l'art	11
3.1	Introduction	11
3.2	Travaux connexes et solutions existantes	11
3.2.1	Solutions open source	11
3.2.2	Solutions propriétaires	11
3.3	Analyse comparative	12
3.3.1	Atouts et lacunes identifiés	12
3.4	Conclusion	12
4	Analyse et conception	13
4.1	Introduction	13
4.2	Formalisme de modélisation (UML)	13
4.3	Architecture logicielle	13
4.3.1	Approche architecturale	13
4.3.2	Schéma de l'architecture et organisation en couches	14
4.3.3	Discussion	14
4.4	Diagrammes de conception	15
4.4.1	Diagramme de classes	15
4.4.2	Diagrammes de séquence	16
4.5	Modélisation de la base de données	17
4.5.1	Schéma relationnel	17
4.5.2	Contraintes d'intégrité	17
4.6	Conclusion	17
5	Technologies et réalisation	19
5.1	Introduction	19
5.2	Choix techniques et outils	19
5.2.1	Langages, Frameworks, Bases de données	19
5.2.2	Outils de développement	20
5.3	Architecture de déploiement	20
5.3.1	Environnement cible	20
5.3.2	Pipeline CI/CD	20
5.4	Implémentation et interfaces	21
5.4.1	Structure du code backend (Node.js)	21
5.4.2	Structure du code web-admin (React)	22
5.4.3	Structure du code mobile (Android)	25
5.4.4	Interfaces utilisateur	27
5.4.5	Fonctionnalités principales implémentées	36
5.5	Stratégie de tests	36
5.5.1	Tests unitaires	36
5.5.2	Tests d'intégration	36
5.5.3	Tests système	37
5.5.4	Outils de test	37
5.5.5	Résultats des tests	37
5.6	Conclusion	37
	Conclusion générale	39
	Apport du projet	41
	Bibliographie	43
	Bibliographie	44

Sitographie	45
Appendices	46
A Annexes (Facultatif)	46

Introduction Générale

Dans le cadre du module Développement Web et Mobile 2, ce projet vise à concevoir et réaliser une plateforme complète de gestion de maintenance des équipements au sein des salles de cours et laboratoires de l'ENS Marrakech. La maintenance des équipements informatiques et matériels représente un enjeu majeur pour les établissements d'enseignement, où la disponibilité des ressources technologiques est essentielle à la qualité de la formation et au bon déroulement des activités pédagogiques.

Actuellement, la gestion des pannes repose souvent sur des procédures manuelles telles que les fiches papier, les courriels ou les appels téléphoniques, qui manquent d'efficacité et de traçabilité. Les signalements peuvent être perdus ou mal suivis, entraînant des délais de réparation excessifs et une frustration chez les utilisateurs qui se trouvent dans l'impossibilité de connaître l'état d'avancement de leurs demandes. Face à ce constat, il est nécessaire de concevoir une solution permettant une gestion fluide, transparente et collaborative des demandes de maintenance. La problématique principale réside dans l'absence d'un système centralisé et efficace pour signaler rapidement une panne sur un équipement spécifique, suivre l'évolution du traitement d'une demande de maintenance, assurer une traçabilité complète des actions effectuées sur chaque ticket, et permettre aux techniciens et administrateurs de gérer l'ensemble des requêtes. Pour répondre à ces besoins, l'objectif principal de ce projet est de développer une plateforme de gestion de maintenance permettant aux utilisateurs de signaler facilement une panne en sélectionnant un équipement, en décrivant le problème et en choisissant un niveau de priorité, tout en offrant aux techniciens une vue d'ensemble des tickets à traiter avec la possibilité de modifier leur statut, d'ajouter des commentaires et d'assigner des ressources, et en fournissant aux administrateurs une interface de supervision complète incluant un tableau de bord avec des statistiques ainsi que la gestion des utilisateurs et des équipements.

Pour répondre à ces objectifs, une architecture orientée services a été adoptée. Le backend est constitué d'une API REST développée avec Node.js et Express, utilisant JWT pour l'authentification et MySQL pour le stockage des données. L'application mobile est développée en Kotlin avec Retrofit pour les appels API, Coroutines pour la programmation asynchrone, et Material Design pour une interface utilisateur moderne. L'interface web est réalisée avec React et Tailwind CSS, offrant un tableau de bord interactif avec des graphiques de statistiques. La base de données respecte la contrainte pédagogique du projet avec quatre tables : users, equipments, maintenance_tickets et ticket_logs, permettant une traçabilité complète des actions effectuées sur chaque ticket.

À l'issue de ce projet, une plateforme fonctionnelle est obtenue, permettant une gestion centralisée et efficace des tickets de maintenance, une réduction des délais de traitement grâce à un suivi en temps réel, une traçabilité complète via l'historique des actions, et une expérience utilisateur moderne et intuitive sur mobile comme sur le web.

Le présent rapport est structuré en six chapitres principaux. Le premier chapitre présente le contexte général du projet ainsi que l'analyse des besoins fonctionnels et non fonctionnels. Le deuxième chapitre est consacré à l'étude des prérequis technologiques nécessaires à la réalisation du projet. Le troisième chapitre présente un état de l'art des solutions existantes et une analyse comparative. Le quatrième chapitre détaille la conception de la solution, incluant les diagrammes

UML, l'architecture logicielle et le schéma de la base de données. Le cinquième chapitre décrit la réalisation technique de la plateforme, couvrant le développement du backend, de l'application mobile et de l'interface web. Enfin, le sixième chapitre présente les résultats des tests effectués, une discussion sur les performances, ainsi qu'une conclusion générale dressant le bilan du travail réalisé et ouvrant des perspectives d'amélioration.

Cette introduction a été validée par l'encadrant du projet, Pr. Mohamed Lachgar, dans le cadre du module Développement Web et Mobile 2.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre a pour vocation de situer le projet dans son ensemble et de préciser les différents paramètres qui en encadrent la réalisation. Il permet de comprendre le lien avec les développements antérieurs ainsi que la façon dont il s'inscrit dans la continuité des enseignements du module Développement Web et Mobile 2.

Rappel du contexte :

La gestion de la maintenance des équipements dans les établissements d'enseignement est une problématique récurrente. Les salles de cours et laboratoires sont équipées de nombreux appareils (ordinateurs, vidéoprojecteurs, imprimantes, tableaux interactifs, etc.) dont la défaillance peut perturber le bon déroulement des séances pédagogiques. Actuellement, le signalement des pannes se fait souvent de manière informelle (papier, email, bouche-à-oreille), ce qui entraîne une perte d'informations et des délais de traitement excessifs.

Objectif du chapitre :

Ce chapitre vise à décrire la nature du projet, sa finalité ainsi que le champ d'applications qu'il couvre. Les informations présentées orientent et justifient les choix techniques à venir. Elles sont également nécessaires pour comprendre les motivations et la portée du travail réalisé.

1.2 Présentation du projet

1.2.1 Sujet du projet

Le projet consiste à développer une plateforme complète de gestion de maintenance des équipements, composée de trois parties distinctes :

- Une **application mobile Android** pour le signalement et le suivi des pannes.
- Une **API REST** backend développée avec Node.js pour la gestion des données et l'authentification.
- Une **interface web d'administration** réalisée avec React pour la supervision et le reporting.

Origine du projet :

Ce projet s'inscrit dans le cadre académique du module Développement Web et Mobile 2 à l'ENS Marrakech. Il répond à un besoin concret constaté au sein des établissements d'enseignement : l'absence d'un outil centralisé pour la gestion des pannes et la traçabilité des interventions.

Domaine d'application et utilisateurs cibles :

Les principaux bénéficiaires sont :

- Les **utilisateurs** (étudiants, enseignants, personnel) qui signalent les pannes.
- Les **techniciens** qui traitent les demandes et interviennent sur les équipements.

- Les **administrateurs** qui supervisent l'ensemble du processus et gèrent les équipements.

1.2.2 Intérêt du projet

Améliorations apportées :

La plateforme proposée apporte plusieurs améliorations par rapport aux méthodes traditionnelles :

- Centralisation des demandes de maintenance.
- Traçabilité complète des actions (historique des tickets).
- Réduction des délais de traitement grâce à une priorisation des demandes.
- Interface moderne et intuitive accessible depuis mobile et web.

Impact et pertinence :

Ce projet est pertinent car il répond à un besoin réel : la digitalisation du processus de maintenance. Il permet de gagner en efficacité, en transparence et en réactivité. De plus, il respecte la contrainte pédagogique d'un maximum de quatre tables dans la base de données, tout en couvrant l'ensemble des fonctionnalités attendues.

1.3 Problématique et état de l'existant

Solutions existantes :

Plusieurs solutions de gestion de maintenance existent sur le marché (GLPI, OTRS, Jira Service Management, etc.). Cependant, ces solutions sont souvent :

- Complexes et lourdes à déployer.
- Payantes ou nécessitant des licences.
- Non adaptées aux petits établissements (surcharge fonctionnelle).

Défauts et limites :

Les solutions existantes ne proposent pas toujours :

- Une application mobile dédiée pour le signalement rapide des pannes.
- Une interface de suivi simplifiée pour les utilisateurs non techniques.
- Une traçabilité complète sous forme d'historique des actions.

Ces lacunes justifient la nécessité de développer une solution sur mesure, adaptée aux besoins spécifiques des établissements d'enseignement.

1.4 Objectifs du projet

1.4.1 Objectifs fonctionnels

- Permettre aux utilisateurs de créer un ticket de maintenance (équipement, description, priorité).
- Permettre aux techniciens de consulter et modifier le statut des tickets.
- Permettre aux administrateurs de gérer les équipements et les utilisateurs.
- Offrir un historique complet des actions sur chaque ticket.
- Fournir un tableau de bord avec des statistiques (nombre de tickets, répartition par priorité, etc.).

1.4.2 Objectifs techniques

- Développer une API REST sécurisée avec JWT.
- Assurer le hashage des mots de passe avec bcrypt.
- Respecter une architecture MVC.
- Assurer la compatibilité de l'application Android (API 24+).
- Garantir un temps de réponse de l'API inférieur à 2 secondes.

1.4.3 Contraintes

- **Temporelles** : le projet doit être livré avant la fin du semestre (environ 12 semaines).
- **Pédagogiques** : maximum 4 tables dans la base de données.
- **Techniques** : utilisation de technologies libres (Node.js, React, Kotlin, MySQL).

1.5 Démarche de gestion de projet

1.5.1 Méthodologie

Nous avons choisi d'adopter une méthodologie **agile de type Scrum**. Cette approche permet une grande adaptabilité et une communication fréquente avec l'encadrant.

Rôle des intervenants :

- **Product Owner** : Pr. Mohamed Lachgar (définition des besoins).
- **Équipe de développement** : un étudiant (développement complet).
- **Sprints** : découpage du projet en itérations de 2 à 3 semaines.

La figure 1.1 présente le schéma du cycle Scrum et de ses principales étapes.

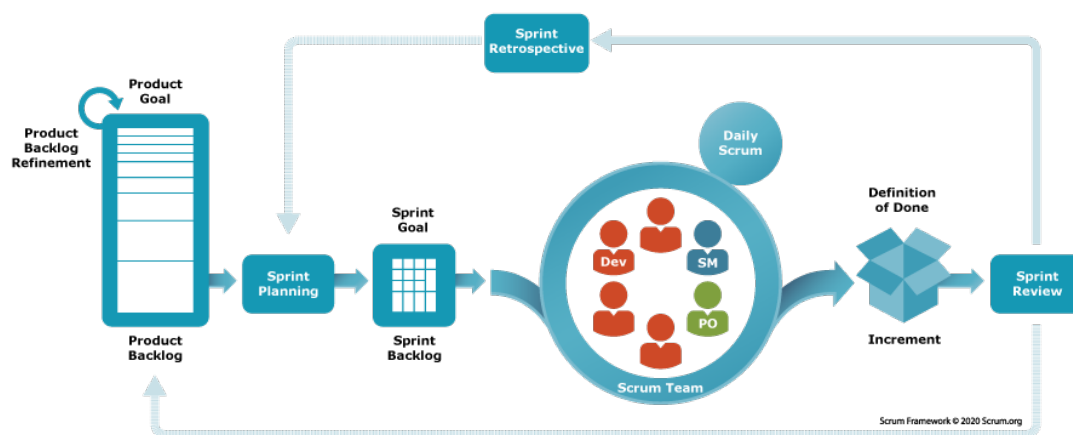


Figure 1.1 : Schéma du cycle Scrum et de ses principales étapes

1.5.2 Planification du projet

Le tableau 1.1 présente le planning détaillé des sprints du projet.

Table 1.1 : Planification des sprints

Sprint	Objectifs	Durée	Livrables
1	Mise en place de l'environnement (Node.js, MySQL, Android Studio) et configuration du dépôt Git.	1 semaine	Environnement configuré, dépôt initial
2	Conception de la base de données (4 tables) et développement de l'API (authentification JWT).	2 semaines	API fonctionnelle, CRUD équipements
3	Développement de l'application mobile Android (connexion, liste des tickets).	2 semaines	Application mobile fonctionnelle
4	Développement de l'interface web React (dashboard, gestion des tickets).	2 semaines	Interface web fonctionnelle
5	Création des tickets (mobile), historique, commentaires et tests d'intégration.	2 semaines	Plateforme complète
6	Optimisation, documentation et préparation de la soutenance.	1 semaine	Version finale, rapport, diaporama

1.6 Conclusion

Ce chapitre a permis de dresser un portrait global du projet. Nous avons présenté les motivations, les objectifs (fonctionnels et techniques), ainsi que les contraintes à respecter. L'état de l'existant a montré les limites des solutions actuelles, justifiant ainsi la nécessité de notre approche.

La méthodologie Scrum et la planification retenue assurent une gestion rigoureuse du projet. Le chapitre suivant sera consacré à l'étude fonctionnelle détaillée, incluant l'identification des acteurs, les cas d'utilisation et les diagrammes UML.

Chapitre 2

Étude préliminaire et fonctionnelle

2.1 Introduction

Ce chapitre a pour objectif de **cadrer les besoins** auxquels le projet doit répondre, qu'ils soient liés aux fonctionnalités principales, aux contraintes techniques ou encore à l'expérience utilisateur. Il permet également de clarifier **l'identité** des différents acteurs, ainsi que leur rôle dans le système. Enfin, il détaille la logique de fonctionnement au travers de cas d'utilisation et de processus métier. Dans le cadre de notre projet de **gestion de maintenance des équipements**, nous identifions trois types d'acteurs : l'utilisateur (déclarant), le technicien et l'administrateur. Chaque acteur dispose de droits et de fonctionnalités spécifiques.

2.2 Étude des besoins

2.2.1 Besoins fonctionnels

Les besoins fonctionnels représentent l'ensemble des services que le système doit offrir aux utilisateurs. Pour notre plateforme, nous distinguons :

- **Gestion des utilisateurs** : inscription, authentification, gestion des rôles (user, tech, admin).
- **Gestion des équipements** : ajout, modification, suppression, consultation et filtrage des équipements (par salle, type, statut).
- **Gestion des tickets** : création, consultation, modification du statut, assignation à un technicien, ajout de commentaires.
- **Suivi et traçabilité** : historique complet des actions sur chaque ticket (logs).
- **Tableau de bord** : statistiques globales (nombre de tickets, répartition par priorité, équipements les plus signalés).

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes techniques et qualitatives du système :

- **Sécurité** : authentification par JWT, hachage des mots de passe avec bcrypt.
- **Performance** : temps de réponse de l'API inférieur à 2 secondes.
- **Disponibilité** : accès 24/7 à la plateforme.
- **Maintenabilité** : code structuré (architecture MVC), documentation claire.
- **Compatibilité** : application mobile Android (API 24+), interface web responsive.
- **Contrainte pédagogique** : maximum 4 tables dans la base de données.

2.3 Identification des acteurs

- **Utilisateur (déclarant)** : personne qui signale une panne. Il peut créer un ticket, consulter ses propres tickets et suivre leur évolution.
- **Technicien** : personne chargée de traiter les pannes. Il peut consulter tous les tickets, changer leur statut, ajouter des commentaires techniques.
- **Administrateur** : responsable de la gestion globale. Il peut gérer les équipements, les utilisateurs, assigner des techniciens et accéder au tableau de bord statistique.

2.4 Cas d'utilisation et diagrammes

2.4.1 Diagramme des cas d'utilisation

La figure [2.1](#) présente le diagramme UML des cas d'utilisation de notre système.

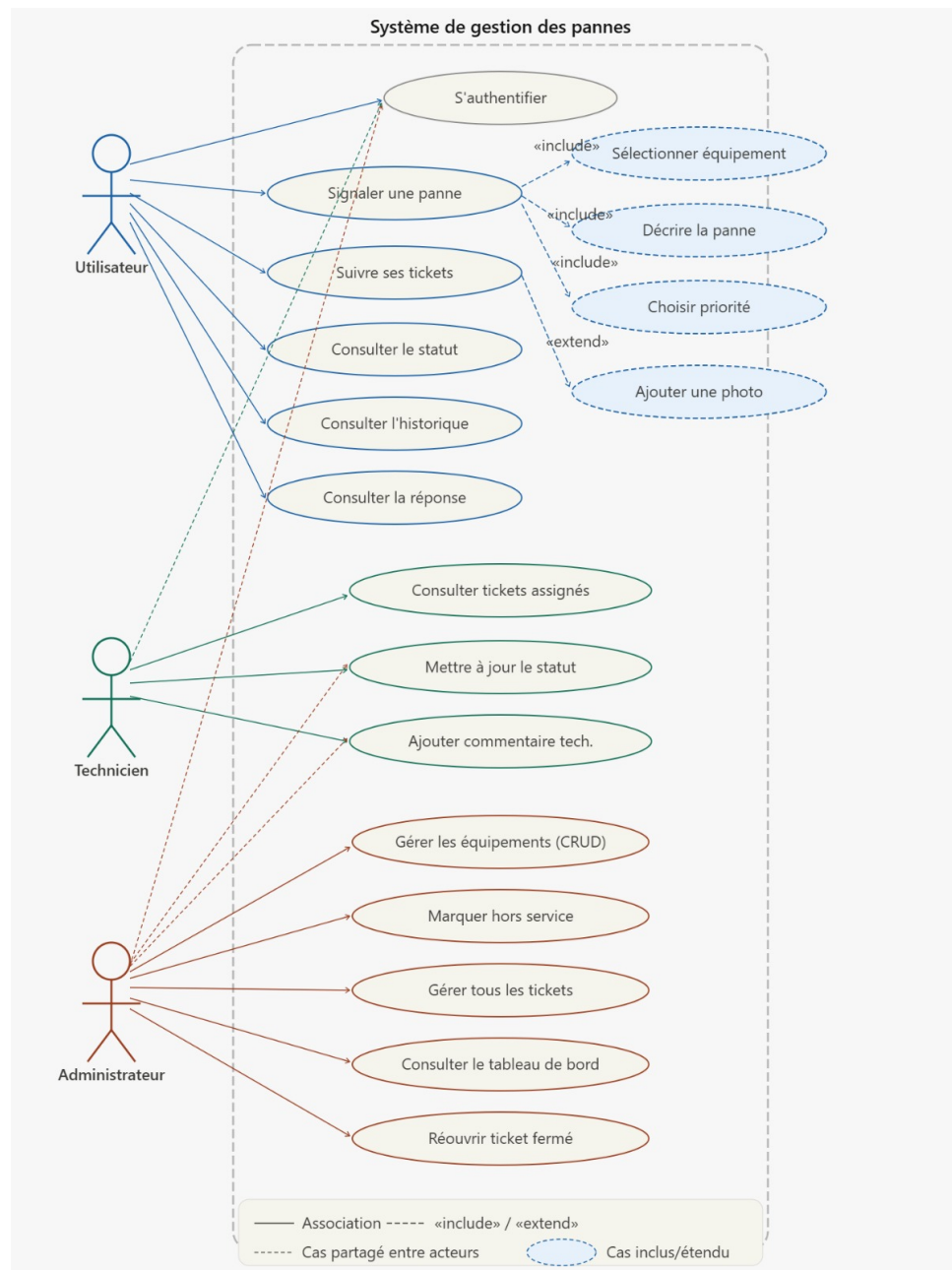


Figure 2.1 : Diagramme des cas d'utilisation de la plateforme de maintenance

2.4.2 Description textuelle des cas d'utilisation

Cas d'utilisation 1 : S'authentifier

- **Acteur** : Utilisateur, Technicien, Administrateur
- **Précondition** : L'utilisateur possède un compte valide.
- **Flux normal** : L'utilisateur saisit son email et mot de passe, le système vérifie les identifiants et génère un token JWT.
- **Postcondition** : L'utilisateur accède à son espace personnel.

Cas d'utilisation 2 : Créer un ticket

- **Acteur** : Utilisateur
- **Précondition** : L'utilisateur est authentifié.

- **Flux normal** : L'utilisateur sélectionne un équipement, décrit la panne, choisit une priorité et valide. Le système enregistre le ticket avec le statut "ouvert".
- **Postcondition** : Un nouveau ticket est créé avec un identifiant unique.

Cas d'utilisation 3 : Consulter les tickets

- **Acteur** : Utilisateur, Technicien, Administrateur
- **Précondition** : L'utilisateur est authentifié.
- **Flux normal** : Le système affiche la liste des tickets (filtrée selon le rôle).
- **Postcondition** : L'utilisateur visualise les tickets.

Cas d'utilisation 4 : Changer le statut d'un ticket

- **Acteur** : Technicien, Administrateur
- **Précondition** : Le ticket existe.
- **Flux normal** : Le technicien sélectionne un nouveau statut (open, in_progress, resolved, closed). Le système met à jour le ticket et ajoute un log.
- **Postcondition** : Le statut du ticket est modifié.

Cas d'utilisation 5 : Gérer les équipements

- **Acteur** : Administrateur
- **Précondition** : L'administrateur est authentifié.
- **Flux normal** : L'administrateur peut ajouter, modifier ou supprimer un équipement.
- **Postcondition** : La base des équipements est mise à jour.

2.5 Processus métier

La figure ?? illustre le déroulement complet du processus de gestion d'un ticket, depuis sa création par un utilisateur jusqu'à sa fermeture par un technicien.

Les étapes clés du processus sont :

1. L'utilisateur crée un ticket (ouverture).
2. Le ticket est assigné à un technicien par l'administrateur.
3. Le technicien change le statut en "en cours" et ajoute un commentaire.
4. Le technicien résout la panne et change le statut en "résolu".
5. L'utilisateur peut consulter la solution et clôturer le ticket (fermé).

2.6 Conclusion

Ce chapitre a permis de définir le cadre fonctionnel de notre projet. Nous avons identifié trois acteurs principaux (utilisateur, technicien, administrateur) et listé les besoins fonctionnels et non fonctionnels. Les diagrammes de cas d'utilisation et le processus métier offrent une vision claire du fonctionnement attendu de la plateforme.

Le chapitre suivant sera consacré à l'architecture technique et aux choix technologiques retenus pour la réalisation du projet.

Chapitre 3

État de l'art

3.1 Introduction

Le présent chapitre vise à positionner le projet dans son contexte scientifique et technique, en s'appuyant sur un examen des approches et des travaux déjà réalisés dans le domaine de la gestion de maintenance. Il a pour rôle de **situer votre démarche** par rapport aux solutions existantes et d'aider à **justifier la pertinence** de votre proposition. Les éléments abordés incluent une recherche bibliographique, une comparaison d'outils déjà disponibles, et une mise en exergue des besoins encore non satisfaits.

3.2 Travaux connexes et solutions existantes

Dans cette section, nous recensons et décrivons les principaux acteurs ou contributions pertinentes pour la gestion de maintenance.

3.2.1 Solutions open source

- **GLPI (Gestionnaire Libre de Parc Informatique)** : GLPI est une solution open source mature de gestion des services informatiques (ITSM). Elle permet la gestion des tickets, des équipements et des contrats. **Points forts** : communauté active, riche fonctionnelle, modularité. **Limites** : interface vieillissante, complexité pour les petits établissements, absence d'application mobile native.
- **OTRS (Open Ticket Request System)** : OTRS est un système de gestion de tickets utilisé principalement pour le support client. **Points forts** : flexibilité, personnalisation avancée, gestion des flux de travail. **Limites** : lourd à déployer, courbe d'apprentissage élevée, coût pour certaines fonctionnalités.
- **OSTicket** : OSTicket est un système de tickets léger et simple à installer. **Points forts** : gratuit, facile à prendre en main. **Limites** : fonctionnalités limitées, peu de reporting, design obsolète.

3.2.2 Solutions propriétaires

- **Jira Service Management (Atlassian)** : Solution professionnelle de gestion de services IT. **Points forts** : intégration avec d'autres outils Atlassian, reporting avancé, API robuste. **Limites** : coût élevé (licences payantes), surcharge fonctionnelle pour un besoin simple.
- **ServiceNow** : Plateforme cloud de gestion des services IT. **Points forts** : très complète, automates puissants, conformité RGPD. **Limites** : très coûteuse, réservée aux grandes entreprises, nécessite une équipe dédiée.

3.3 Analyse comparative

Cette partie consiste à **comparer** les solutions précédemment citées selon divers critères.

Table 3.1 : Tableau comparatif des solutions existantes

Solution	Open Source	Application mobile	Traçabilité	Coût
GLPI	Oui	Non (interface web mobile)	Oui	Gratuit
OTRS	Oui (version CE)	Non	Oui	Gratuit (CE)
OSTicket	Oui	Non	Partielle	Gratuit
Jira Service Management	Non	Oui	Oui	Payant
ServiceNow	Non	Oui	Oui	Très élevé
Notre solution	Oui	Oui (Android)	Complète (logs)	Gratuit

3.3.1 Atouts et lacunes identifiés

- **Atouts à exploiter** : Les solutions open source existantes (GLPI, OSTicket) offrent une base solide mais souffrent d'un manque de modernité (interface, absence d'application mobile).
- **Lacunes identifiées** :
 - Absence d'application mobile dédiée pour le signalement rapide des pannes. Interfaces utilisateur souvent complexes et peu intuitives.
 - Manque de traçabilité complète (historique des actions).
 - Reporting limité (peu ou pas de graphiques).

3.4 Conclusion

Pour conclure ce chapitre, il est essentiel de faire une **mise en perspective** :

- **Bilan de l'analyse** : Les solutions existantes (GLPI, OTRS, OSTicket, Jira, ServiceNow) présentent des limites importantes pour notre contexte : absence d'application mobile, coût élevé, complexité ou interfaces obsolètes.
- **Orientation** : Cette étude confirme la pertinence de développer une solution sur mesure, spécifiquement adaptée aux besoins des établissements d'enseignement. Notre approche se distingue par :
 - Une **application mobile Android** pour le signalement rapide.
 - Une **API REST sécurisée** (JWT, bcrypt).
 - Une **interface web moderne** (React, Tailwind CSS).
 - Une **base de données légère** (4 tables) respectant la contrainte pédagogique.
 - Une **traçabilité complète** via l'historique des actions.
- **Transition vers le chapitre suivant** : Les conclusions tirées ici serviront de base pour **élaborer l'architecture technique**, définir la modélisation UML (diagrammes de classes, de cas d'utilisation, de séquence) et sélectionner les outils de développement appropriés (Node.js, React, Kotlin, MySQL).

Ainsi, le chapitre «État de l'art» fournit un **cadre clair** des approches préexistantes et met en relief la **valeur ajoutée** que notre système propose. Il pave la voie au prochain chapitre, qui se concentrera sur la **conception** et l'architecture de la solution.

Chapitre 4

Analyse et conception

4.1 Introduction

Ce chapitre a pour objectif de traduire les besoins fonctionnels et non fonctionnels du projet en une solution technique détaillée. Il explique comment les choix de modélisation, d'architecture et de conception ont été déterminés et justifiés, et comment ils s'inscrivent dans la logique globale du projet.

Notre système repose sur une architecture orientée services, composée de trois entités principales : une application mobile Android (client), une API REST (backend) et une interface web d'administration (React). La base de données relationnelle MySQL assure la persistance des données.

4.2 Formalisme de modélisation (UML)

Motivation et choix

Nous avons choisi d'utiliser le langage de modélisation **UML (Unified Modeling Language)** pour décrire la structure et le comportement du système. Ce formalisme permet de :

- Visualiser clairement les entités et leurs relations.
- Communiquer efficacement les choix de conception.
- Garantir la cohérence et la maintenabilité du projet.

Diagrammes retenus

- **Diagramme de classes** : pour représenter la structure statique (entités, attributs, méthodes, relations).
- **Diagrammes de séquence** : pour illustrer l'enchaînement des interactions lors des scénarios clés (connexion, création de ticket, changement de statut).
- **Diagramme de composants** : pour visualiser l'organisation modulaire du système.
- **Diagramme de déploiement** : pour illustrer la répartition des composants sur l'infrastructure (client mobile, serveur web, base de données).

4.3 Architecture logicielle

4.3.1 Approche architecturale

Nous avons opté pour une architecture **orientée services** (API REST), qui se situe entre l'architecture monolithique et les microservices.

- **Architecture monolithique** : *Avantages* : simplicité de développement initial, déploiement unifié. *Inconvénients* : difficulté de maintenance à grande échelle.
- **Architecture microservices** : *Avantages* : scalabilité fine, isolation des pannes. *Inconvénients* : complexité de déploiement (non justifiée pour ce projet).
- **Architecture retenue (API REST)** : *Avantages* : séparation claire entre client mobile, web et backend, évolutivité, réutilisabilité.

4.3.2 Schéma de l'architecture et organisation en couches

Le système est divisé en trois couches principales :

- **Couche de présentation** : Application mobile Android (Kotlin) et interface web (React). Responsable de l'interaction utilisateur et de l'affichage des données.
- **Couche de logique métier (API REST)** : Développée avec Node.js et Express. Gère l'authentification (JWT), les règles métier et l'accès aux données.
- **Couche de données** : Base de données MySQL. Responsable de la persistance des informations (utilisateurs, équipements, tickets, logs).

La figure 4.1 présente le schéma global de l'architecture.

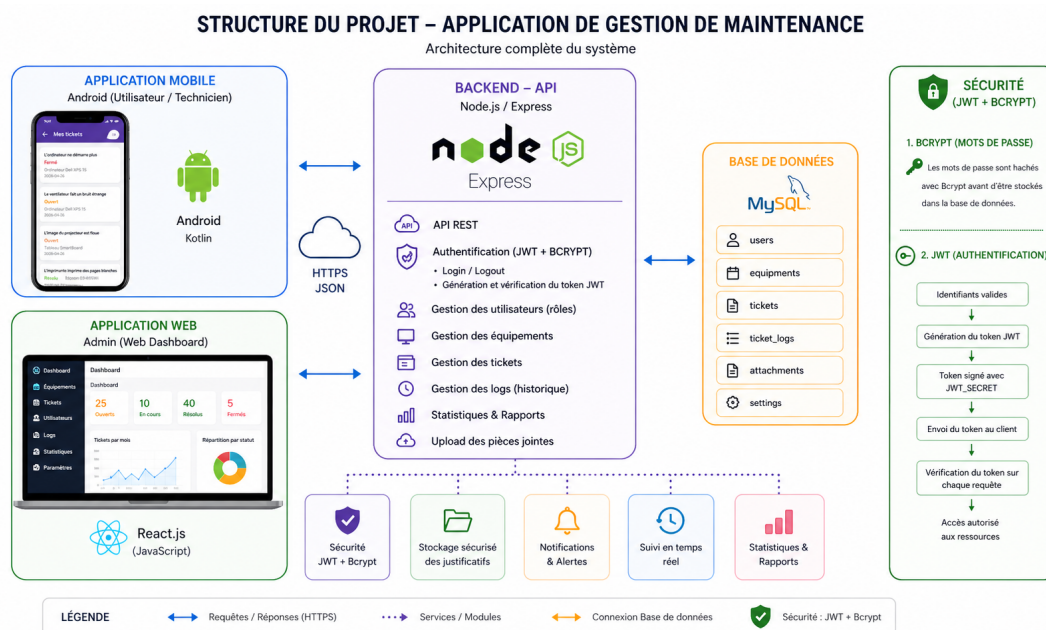


Figure 4.1 : Architecture globale de la plateforme de maintenance

4.3.3 Discussion

Cette architecture répond aux besoins fonctionnels et non fonctionnels identifiés :

- **Maintenabilité** : séparation des couches facilitant les évolutions.
- **Sécurité** : authentification JWT, hash bcrypt.
- **Performance** : API légère, requêtes optimisées.
- **Traçabilité** : table ticket_logs pour l'historique.

4.4 Diagrammes de conception

4.4.1 Diagramme de classes

Le diagramme de classes (figure 4.2) présente la structure statique de la base de données.

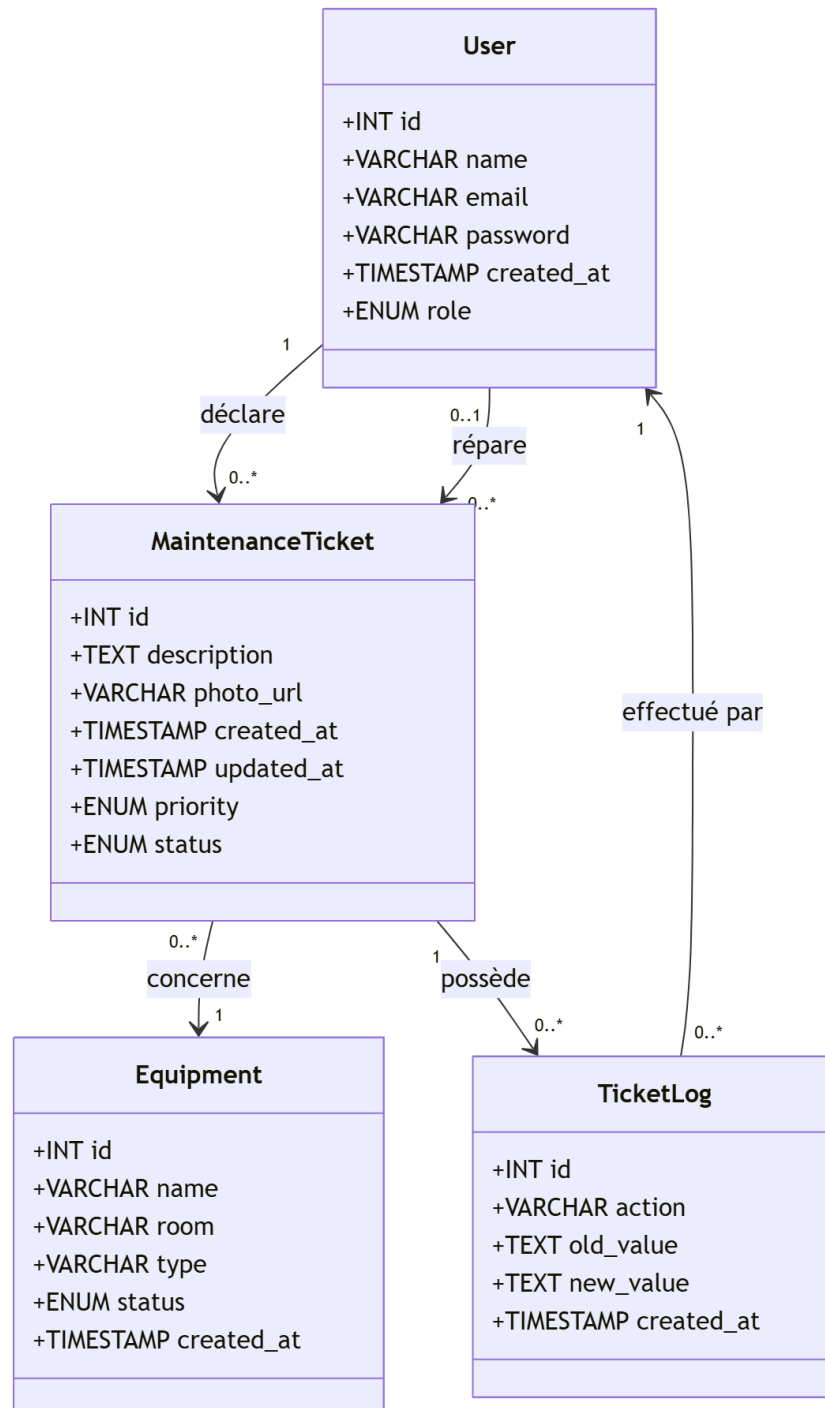


Figure 4.2 : Diagramme de classes de la base de données

Présentation des entités :

- **User** : id, name, email, password, role, created_at.
- **Equipment** : id, name, room, type, status, created_at.

- Ticket : id, equipment_id (FK), user_id (FK), technician_id (FK), description, priority, status, photo_url, resolution_comment, created_at, updated_at.
- TicketLog : id, ticket_id (FK), user_id (FK), action, old_value, new_value, created_at.

Relations et cardinalités :

- Un User peut créer plusieurs Ticket (1,n).
- Un Equipment peut être associé à plusieurs Ticket (1,n).
- Un Ticket peut avoir plusieurs TicketLog (1,n).

4.4.2 Diagrammes de séquence

Scénario 1 : Authentification

La figure 4.3 illustre le flux d'authentification d'un utilisateur.

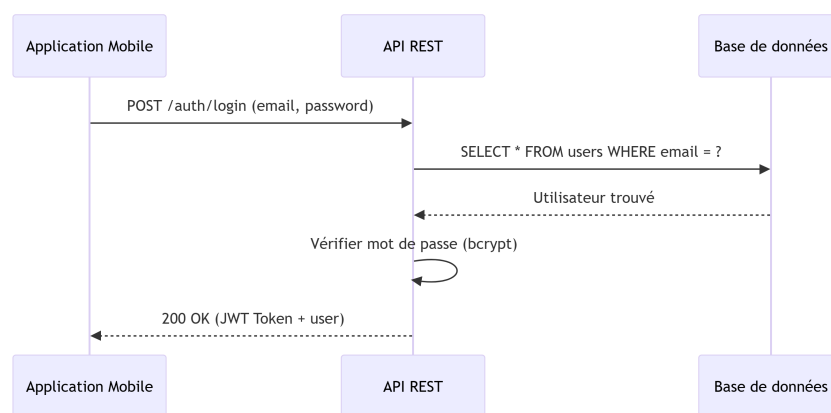


Figure 4.3 : Diagramme de séquence – Authentification

Scénario 2 : Création d'un ticket

La figure 4.4 présente le flux de création d'un ticket depuis l'application mobile.

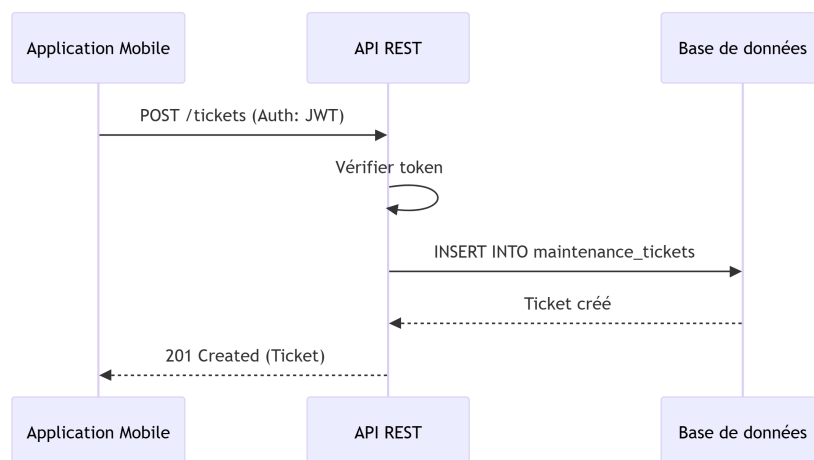


Figure 4.4 : Diagramme de séquence – Création d'un ticket

Scénario 3 : Changement de statut

La figure 4.5 montre le processus de mise à jour du statut par un technicien.

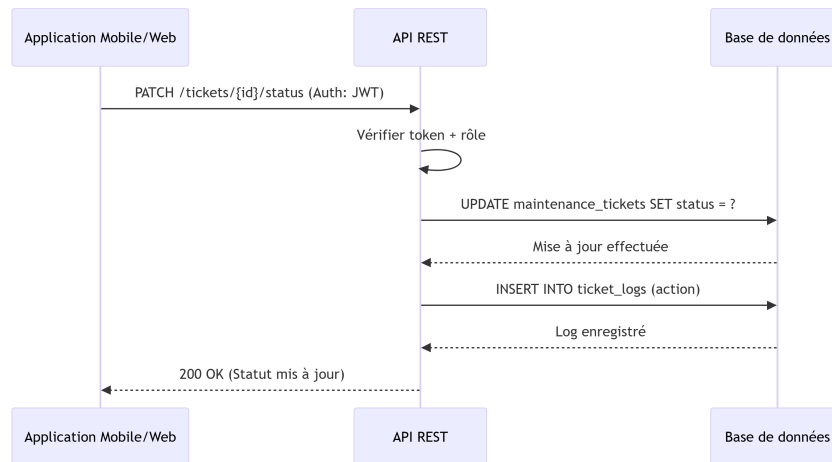


Figure 4.5 : Diagramme de séquence – Changement de statut

4.5 Modélisation de la base de données

La base de données respecte la contrainte pédagogique de 4 tables maximum.

4.5.1 Schéma relationnel

- **users** (id, name, email, password, role, reset_token, reset_expires, created_at)
- **equipments** (id, name, room, type, status, created_at)
- **maintenance_tickets** (id, equipment_id, user_id, technician_id, description, priority, status, photo_url, resolution_comment, created_at, updated_at)
- **ticket_logs** (id, ticket_id, user_id, action, old_value, new_value, created_at)

4.5.2 Contraintes d'intégrité

- equipment_id référence equipments(id) (ON DELETE CASCADE).
- user_id référence users(id) (ON DELETE CASCADE).
- technician_id référence users(id) (ON DELETE SET NULL).
- ticket_id dans ticket_logs référence maintenance_tickets(id).

4.6 Conclusion

Pour conclure ce chapitre, nous récapitulons les éléments essentiels présentés :

- **Synthèse des choix d'architecture et de modélisation** : Nous avons opté pour une architecture orientée services (API REST) avec trois couches (présentation, logique métier, données). Le formalisme UML a permis de modéliser les diagrammes de classes, de séquence, de composants et de déploiement.
- **Justification des décisions** : Ces choix répondent aux besoins fonctionnels (gestion des tickets, traçabilité) et non fonctionnels (sécurité JWT, performance, maintenabilité). La base de données respecte la contrainte pédagogique de 4 tables.

- **Transition vers la phase de réalisation** : Les modèles présentés serviront de base pour l'implémentation effective du système, qui sera détaillée dans le chapitre suivant (réalisation technique).

Ce chapitre d'analyse et de conception constitue ainsi le socle technique sur lequel reposera la réalisation du projet.

Chapitre 5

Technologies et réalisation

Ce chapitre décrit l'ensemble des technologies, outils et méthodes qui ont été utilisés pour la mise en œuvre du projet. Il présente d'abord les choix techniques, puis détaille l'environnement de déploiement, l'implémentation du système (structure du code et interfaces utilisateur), ainsi que la stratégie de tests appliquée. Enfin, une conclusion synthétique fait le point sur les réalisations et les éventuelles difficultés rencontrées.

5.1 Introduction

Dans cette section introductive, nous donnons une vue d'ensemble des aspects techniques et pratiques abordés dans ce chapitre. Notre projet se compose de trois parties distinctes :

- Un **backend** (API REST) développé avec Node.js et Express.
- Une **application mobile Android** développée en Kotlin.
- Une **interface web d'administration** développée avec React et Tailwind CSS.
- Une **base de données MySQL** pour la persistance des données.

Ces choix technologiques ont été guidés par les besoins fonctionnels (gestion des tickets, traçabilité) et les contraintes techniques (sécurité, performance, contrainte pédagogique de 4 tables).

5.2 Choix techniques et outils

5.2.1 Langages, Frameworks, Bases de données

- **Backend (API REST)** : Nous avons choisi **Node.js** avec le framework **Express** pour sa légèreté, sa grande communauté et sa facilité d'intégration avec MySQL. L'authentification est assurée par **JWT (JSON Web Token)** et le hashage des mots de passe par **bcrypt**.
- **Application mobile Android** : Développée en **Kotlin**, le langage moderne recommandé par Google pour Android. Les bibliothèques utilisées sont :
 - **Retrofit** : pour les appels HTTP vers l'API REST.
 - **Coroutines** : pour la programmation asynchrone.
 - **ViewModel et LiveData** : pour l'architecture MVVM.
 - **RecyclerView et CardView** : pour l'affichage des listes.
 - **Material Design** : pour une interface utilisateur moderne.
- **Interface web** : Développée avec **React** (bibliothèque JavaScript) et **Tailwind CSS** pour le style. Les graphiques sont réalisés avec **Chart.js**.
- **Base de données** : Nous avons opté pour **MySQL**, une base de données relationnelle robuste et adaptée aux besoins de persistance du projet. La base contient 4 tables : `users`, `equipments`, `maintenance_tickets`, `ticket_logs`.

- **Critères de sélection** : Les critères ayant guidé ces choix sont : simplicité de mise en œuvre, performance, documentation abondante, compatibilité multiplateforme et respect de la contrainte pédagogique.

5.2.2 Outils de développement

- **IDE** :
 - **Android Studio** : pour le développement de l'application mobile.
 - **Visual Studio Code** : pour le backend (Node.js) et l'interface web (React).
- **Gestion de versions** : **Git** avec un dépôt hébergé sur **GitHub** pour le suivi des modifications et la gestion du code source.
- **Gestionnaire de paquets** :
 - **npm** pour Node.js.
 - **Gradle** pour Android.
 - **npm** pour React.
- **Outils de test** :
 - **Postman** : pour les tests d'API.
 - **Logcat** : pour le débogage Android.

5.3 Architecture de déploiement

5.3.1 Environnement cible

Le système est déployé sur une architecture locale (environnement de développement) :

- **Backend** : serveur Node.js tournant sur le port 3000.
- **Base de données** : MySQL sur le même serveur (localhost).
- **Application mobile** : exécutée sur émulateur Android (ou appareil physique).
- **Interface web** : serveur React tournant sur le port 3001.

Pour un déploiement en production, on pourrait envisager :

- Hébergement du backend sur une instance cloud (AWS EC2, Heroku, etc.).
- Hébergement de la base de données sur un service comme AWS RDS.
- Conteneurisation avec Docker pour faciliter le déploiement.

5.3.2 Pipeline CI/CD

Une intégration continue simplifiée a été mise en place via Git :

- Chaque commit poussé sur la branche principale déclenche une vérification manuelle.
- Les tests sont exécutés localement avant chaque push.

L'automatisation complète (CI/CD) pourra être ajoutée ultérieurement (GitHub Actions, GitLab CI).

5.4 Implémentation et interfaces

5.4.1 Structure du code backend (Node.js)

L'API backend est développée avec **Node.js** et le framework **Express**. Le code est organisé selon une architecture **MVC (Model-View-Controller)** simplifiée, où les modèles sont gérés via la base de données MySQL et les requêtes SQL préparées.

L'organisation du projet est la suivante :

```
backend/  
  server.js  
  package.json  
  .env  
  config/  
    db.js  
  controllers/  
    authController.js  
    equipmentController.js  
    ticketController.js  
    dashboardController.js  
  middleware/  
    auth.js  
  routes/  
    auth.js  
    equipments.js  
    tickets.js  
    dashboard.js  
  scripts/  
    hashPasswords.js
```

Description des principaux dossiers et fichiers :

- **server.js** : point d'entrée de l'application. Configure les middlewares (CORS, JSON), les routes et démarre le serveur.
- **config/db.js** : gestion de la connexion à la base de données MySQL via un pool de connexions sécurisé.
- **controllers/** : contient la logique métier de l'application. Chaque contrôleur est responsable d'une entité (authentification, équipements, tickets, tableau de bord).
- **middleware/auth.js** : implémente la vérification du token JWT et la gestion des rôles (utilisateur, technicien, administrateur).
- **routes/** : définit les endpoints de l'API REST et associe chaque route à un contrôleur. Les routes protégées utilisent le middleware d'authentification.
- **scripts/hashPasswords.js** : utilitaire permettant de hacher les mots de passe existants dans la base de données (migration).

Table 5.1 : Principaux endpoints de l'API REST

Méthode	Endpoint	Rôle requis	Description
POST	/api/auth/login	Aucun	Authentification et génération d'un token JWT
POST	/api/auth/register	Aucun	Inscription d'un nouvel utilisateur
GET	/api/equipments	Authentifié	Récupération de la liste des équipements
POST	/api/equipments	Admin	Ajout d'un nouvel équipement
PUT	/api/equipments/:id	Admin	Modification d'un équipement
DELETE	/api/equipments/:id	Admin	Suppression d'un équipement
GET	/api/tickets	Authentifié	Récupération des tickets (filtrage selon le rôle)
POST	/api/tickets	Authentifié	Création d'un nouveau ticket
PATCH	/api/tickets/:id/status	Tech/Admin	Changement du statut d'un ticket
GET	/api/dashboard/stats	Tech/Admin	Statistiques pour le tableau de bord

Principaux endpoints de l'API :

Sécurité et validation :

- Authentification : **JWT (JSON Web Token)** avec une durée de validité de 24 heures.
- Mots de passe : hashés avec **bcrypt** avant stockage en base.
- Requêtes SQL : exécutées de manière préparée pour prévenir les injections SQL.
- Validation des entrées : via des expressions régulières (email, champs requis).
- Gestion des rôles : middleware dédié pour restreindre l'accès aux routes sensibles.

Cette architecture garantit une séparation claire des responsabilités, une évolutivité facilitée et une sécurité renforcée.

5.4.2 Structure du code web-admin (React)

L'interface web d'administration est développée avec la bibliothèque JavaScript **React**. Le code est organisé selon une architecture modulaire basée sur les composants, facilitant ainsi la réutilisation et la maintenabilité.

Organisation du projet :

```
web-admin/
  public/
    index.html
  src/
    App.js
    index.js
    index.css
    components/
      Layout/
        Layout.js
        Sidebar.js
```

```
    Header.js
  Dashboard/
    StatCard.js
    PriorityChart.js
    TopEquipmentsChart.js
    TicketsTrendChart.js
  Common/
    PrivateRoute.js
    LoadingSpinner.js
  pages/
    Login.js
    Dashboard.js
    Equipments.js
    Tickets.js
    ForgotPassword.js
    ResetPassword.js
  services/
    api.js
    auth.js
  context/
    AuthContext.js
package.json
tailwind.config.js
```

Description des dossiers principaux :

- **components/** : contient les composants réutilisables de l'application (Layout, graphiques, éléments communs).
- **pages/** : regroupe les différentes pages de l'application (connexion, tableau de bord, gestion des équipements, gestion des tickets).
- **services/** : contient les fonctions d'appel à l'API REST et la gestion de l'authentification.
- **context/** : implémente le contexte React pour la gestion globale de l'état d'authentification (AuthContext).

Table 5.2 : Principales pages et composants de l'interface web

Composant/Page	Type	Description
Login.js	Page	Gère l'authentification de l'utilisateur (email, mot de passe, stockage du token JWT)
Dashboard.js	Page	Affiche le tableau de bord avec les statistiques (graphiques, indicateurs)
Equipments.js	Page	Gestion des équipements (CRUD complet : ajout, modification, suppression, filtres)
Tickets.js	Page	Gestion des tickets (liste, filtres, changement de statut, assignation technicien)
ForgotPassword.js	Page	Permet à l'utilisateur de demander une réinitialisation de mot de passe
ResetPassword.js	Page	Permet de définir un nouveau mot de passe via un token de réinitialisation
Sidebar.js	Composant	Barre latérale de navigation avec les icônes et le menu
Header.js	Composant	Barre de navigation supérieure avec les informations utilisateur
PrivateRoute.js	Composant	Protège les routes sensibles en vérifiant l'authentification JWT

Rôles des principaux fichiers :

- `api.js` : configuration d'Axios (base URL, intercepteur pour ajouter automatiquement le token JWT).
- `auth.js` : fonctions d'authentification (login, logout, gestion du token et de l'utilisateur dans le `localStorage`).
- `AuthContext.js` : contexte React permettant de partager l'état d'authentification dans toute l'application.
- `tailwind.config.js` : configuration de Tailwind CSS (couleurs personnalisées, police d'écriture).

Fonctionnalités principales implémentées :

- Authentification sécurisée avec JWT (stockage du token dans le `localStorage`).
- Tableau de bord interactif avec graphiques (`Chart.js`) :
 - Répartition des tickets par priorité (camembert).
 - Équipements les plus signalés (diagramme à barres).
 - Évolution des tickets dans le temps (graphique linéaire).
- Gestion complète des équipements (CRUD) avec filtres (par salle, type, statut).
- Gestion complète des tickets (liste, filtres, recherche, changement de statut, assignation technicien).
- Interface moderne et responsive grâce à Tailwind CSS.
- Protection des routes sensibles (seuls les utilisateurs authentifiés peuvent y accéder).
- Gestion des rôles (admin, tech, user) avec restriction des fonctionnalités.

Table 5.3 : Dépendances principales du projet web

Bibliothèque	Version	Utilité
React	18.2.0	Bibliothèque principale pour l'interface utilisateur
React Router DOM	6.20.0	Gestion de la navigation entre les pages
Axios	1.6.0	Client HTTP pour les appels à l'API REST
Chart.js	4.4.0	Création des graphiques statistiques
React Chart.js 2	5.2.0	Intégration de Chart.js avec React
Tailwind CSS	3.4.0	Framework CSS pour le design moderne et responsive
Lucide React	0.292.0	Bibliothèque d'icônes modernes

Principales dépendances utilisées : Cette organisation modulaire et l'utilisation de composants réutilisables garantissent une évolutivité, une maintenabilité et une expérience utilisateur optimale.

5.4.3 Structure du code mobile (Android)

L'application mobile Android est organisée selon une architecture respectant le patron de conception **MVC (Model-View-Controller)** simplifié. Cette séparation permet d'assurer une meilleure maintenabilité, une évolutivité et une clarté du code.

Organisation du projet :

```

app/src/main/java/com/maintenance/app/
  MainActivity.kt
  data/
    api/
      ApiService.kt
      RetrofitClient.kt
      AuthInterceptor.kt
    model/
      User.kt
      Equipment.kt
      Ticket.kt
      TicketLog.kt
      ApiResponse.kt
    repository/
      AuthRepository.kt
      TicketRepository.kt
  ui/
    auth/
      LoginActivity.kt
    main/
      MainActivity.kt
    tickets/
      TicketsFragment.kt
      TicketDetailActivity.kt
      CreateTicketActivity.kt
  adapters/

```

```

TicketAdapter.kt
utils/
    TokenManager.kt
    Constants.kt

```

Description des dossiers principaux :

- **data/** : contient les classes d'accès aux données (API, modèles, dépôts).
- **ui/** : regroupe les activités, fragments et adaptateurs (interface utilisateur).
- **utils/** : contient les utilitaires (gestion du token JWT, constantes).

Table 5.4 : Principales activités et fragments de l'application mobile

Classe	Type	Description
LoginActivity	Activity	Gère l'authentification de l'utilisateur (email, mot de passe, token JWT)
MainActivity	Activity	Activité principale contenant la navigation (bottom navigation)
TicketsFragment	Fragment	Affiche la liste des tickets (RecyclerView) selon le rôle de l'utilisateur
TicketDetailActivity	Activity	Affiche les détails d'un ticket (description, statut, priorité, historique)
CreateTicketActivity	Activity	Permet de créer un nouveau ticket (équipement, description, priorité)
TicketAdapter	Adapter	Adaptateur pour afficher les tickets dans une liste (RecyclerView)

Rôles des classes principales :

- **ApiService.kt** : interface définissant les endpoints de l'API REST.
- **RetrofitClient.kt** : configuration du client Retrofit (requêtes HTTP, intercepteur JWT).
- **AuthInterceptor.kt** : intercepteur ajoutant automatiquement le token JWT dans l'en-tête Authorization.
- **TokenManager.kt** : gestion du stockage sécurisé du token JWT et des informations utilisateur (SharedPreferences).
- **Constants.kt** : contient les constantes du projet (URL de l'API, clés de stockage).
- **TicketAdapter.kt** : adaptateur permettant l'affichage des tickets dans un RecyclerView avec gestion des couleurs selon la priorité et le statut.

Fonctionnalités principales implémentées :

- Authentification sécurisée avec JWT.
- Affichage de la liste des tickets (filtrage selon le rôle : user, tech, admin).
- Création d'un nouveau ticket (équipement, description, priorité).
- Consultation du détail d'un ticket (description, statut, priorité, historique des actions).
- Gestion du token JWT (stockage, ajout automatique aux requêtes).

Cette organisation modulaire facilite la maintenance, l'ajout de nouvelles fonctionnalités et garantit une expérience utilisateur fluide.

5.4.4 Interfaces utilisateur

Application mobile Android

La figure 5.1 montre l'écran de connexion de l'application mobile.

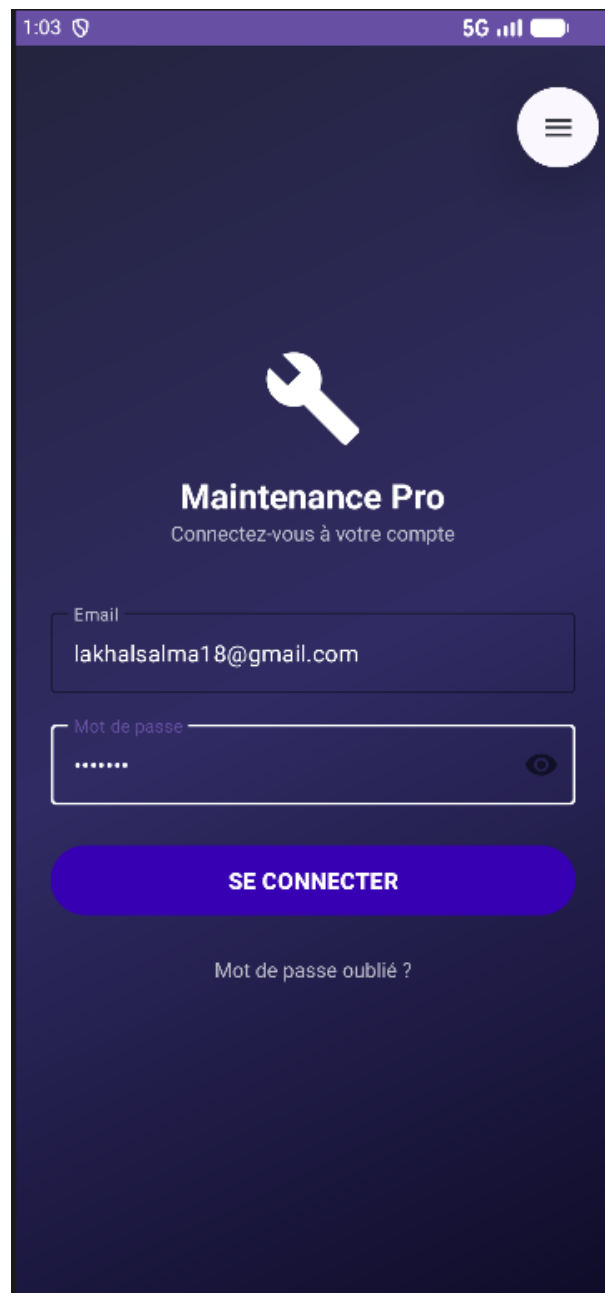


Figure 5.1 : Écran de connexion (Android)

La figure 5.2 présente la liste des tickets.

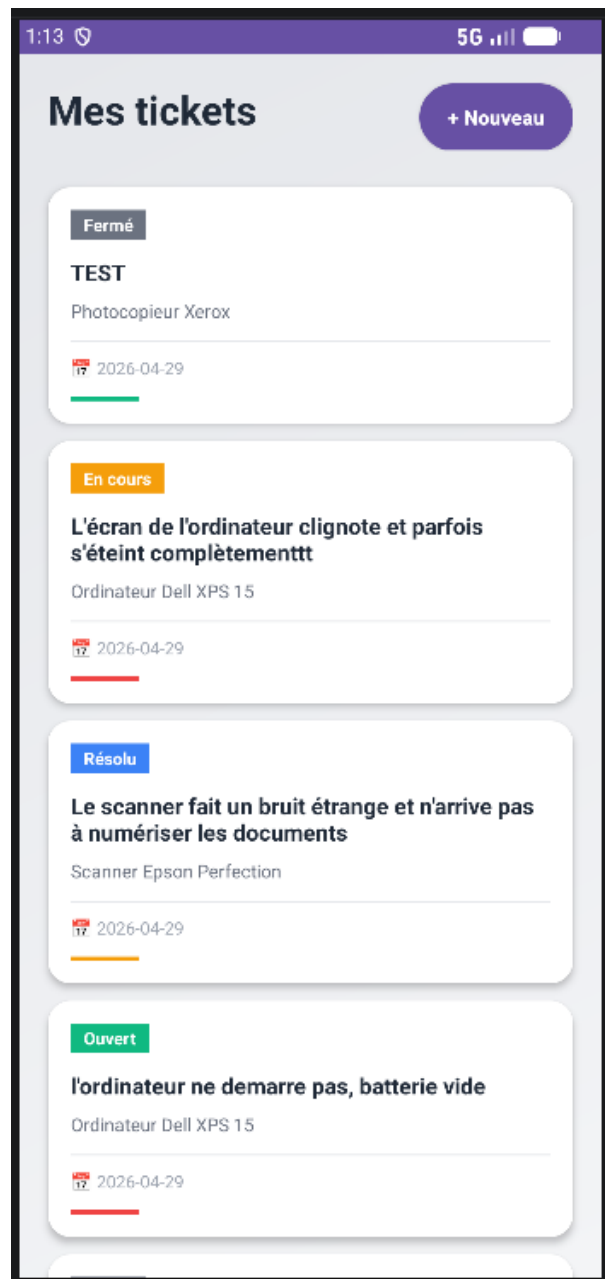


Figure 5.2 : Liste des tickets (Android)

La figure 5.3 montre le formulaire de création d'un ticket.

1:11 5G

← Nouveau ticket

Équipement

Imprimante 3D Creality - Salle A101

Description de la panne

Le tableau tactile ne répond plus au stylo

Priorité

☐ Basse ☐ Moyenne ☒ Haute

SIGNALER LA PANNE

Figure 5.3 : Création d'un ticket (Android)

La figure 5.4 illustre la page de détail d'un ticket avec son historique.

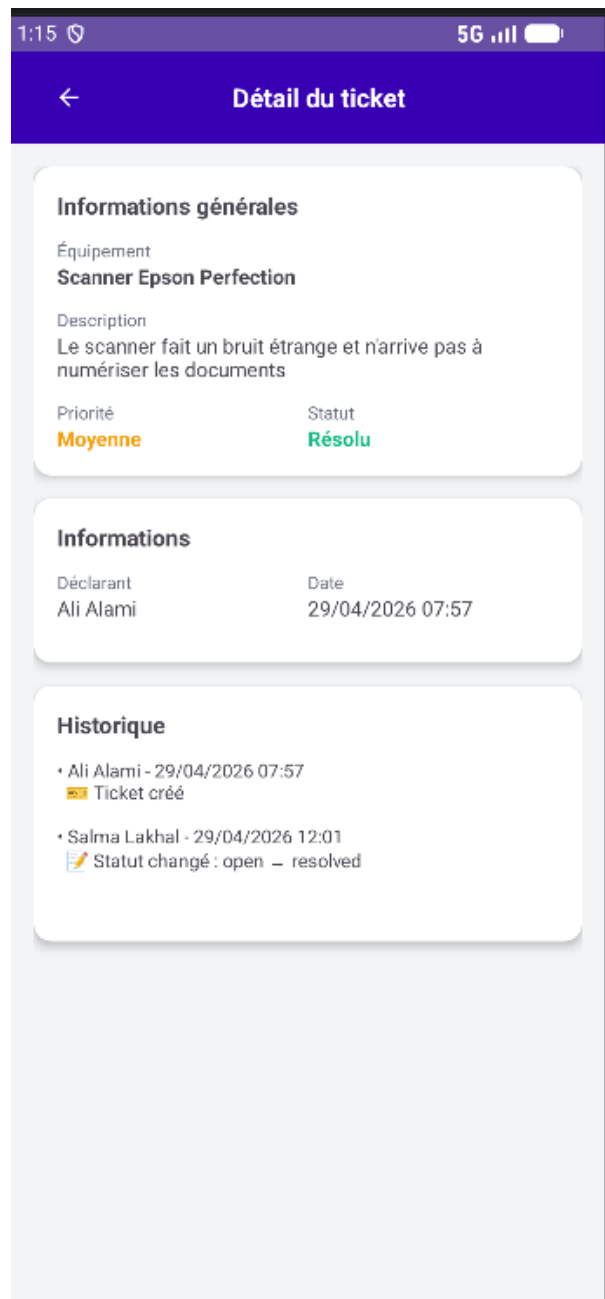


Figure 5.4 : Détail d'un ticket avec historique (Android)

Interface web d'administration

La figure 5.5 présente l'écran de connexion de l'interface web.

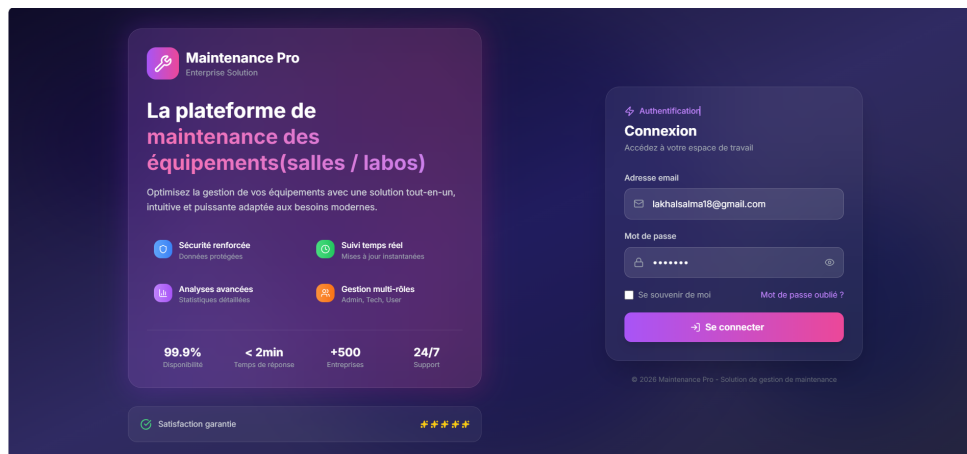


Figure 5.5 : Écran de connexion (Web)

La figure 5.6 montre le tableau de bord avec les statistiques et graphiques.

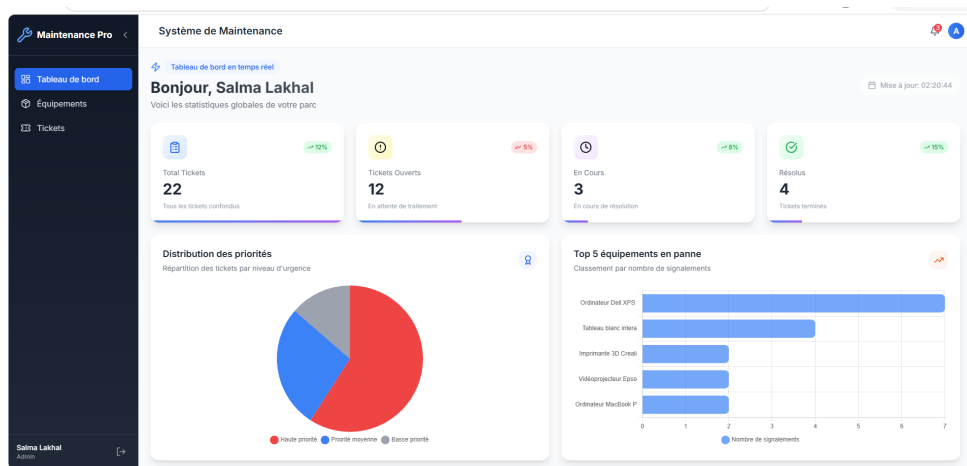


Figure 5.6 : Tableau de bord (Web)

Gestion des équipements (CRUD complet) La figure 5.7 montre la liste complète des équipements avec filtres (par salle, type, statut).

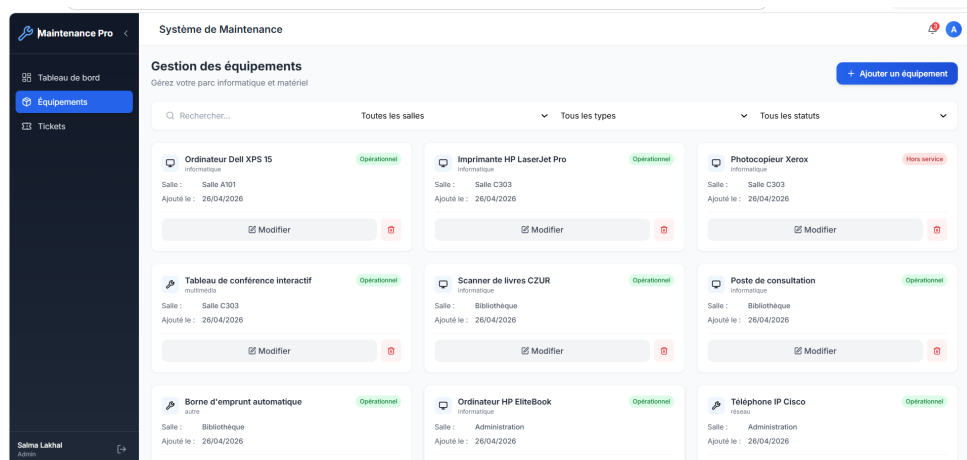


Figure 5.7 : Liste des équipements (Web) – Consultation (Lire)

La figure 5.8 illustre le formulaire d'ajout d'un nouvel équipement.



Ajouter un équipement

Nom de l'équipement *

Salle *

Sélectionner une salle ▼

Type *

Sélectionner un type ▼

Statut

Opérationnel ▼

Annuler Ajouter

Figure 5.8 : Création d'un équipement (Web) – Créer

La figure 5.9 montre un exemple concret de création d'un équipement.



Ajouter un équipement

Nom de l'équipement *

Salle *

Labo B203 ▼

Type *

réseau ▼

Statut

Opérationnel ▼

Annuler Ajouter

Figure 5.9 : Exemple de création d'un équipement (Web)

La figure 5.10 montre l'équipement avant modification.



Figure 5.10 : État de l'équipement avant modification

La figure 5.11 montre le formulaire de modification d'un équipement existant.



Figure 5.11 : Modification d'un équipement (Web) – Modifier

La figure 5.12 montre l'équipement après modification.



Figure 5.12 : État de l'équipement après modification

La figure 5.13 illustre la confirmation de suppression d'un équipement.

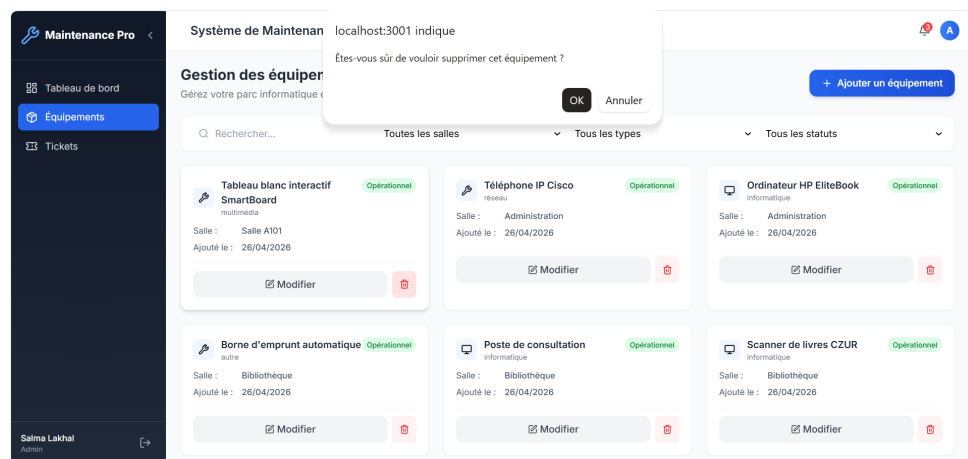


Figure 5.13 : Confirmation de suppression d'un équipement (Web)

La figure 5.14 montre l'interface après suppression de l'équipement.

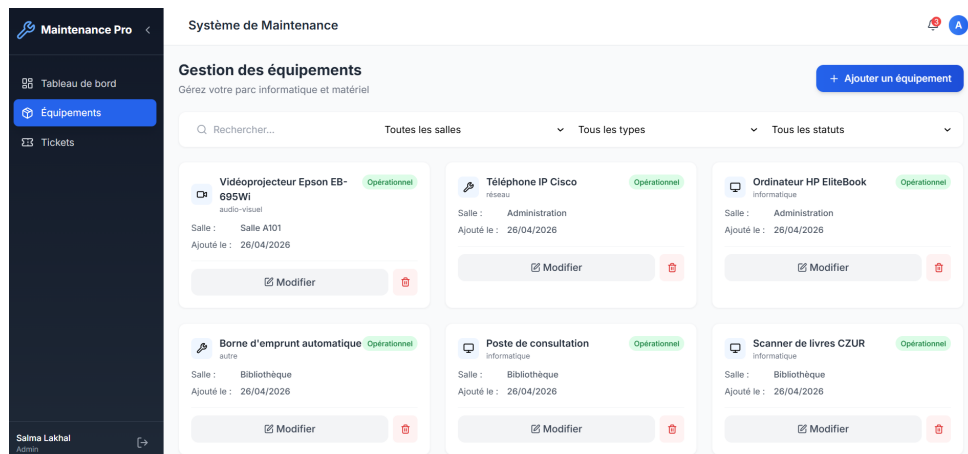


Figure 5.14 : Liste des équipements après suppression

Gestion des tickets La figure 5.15 présente la liste des tickets avec filtres et recherche.

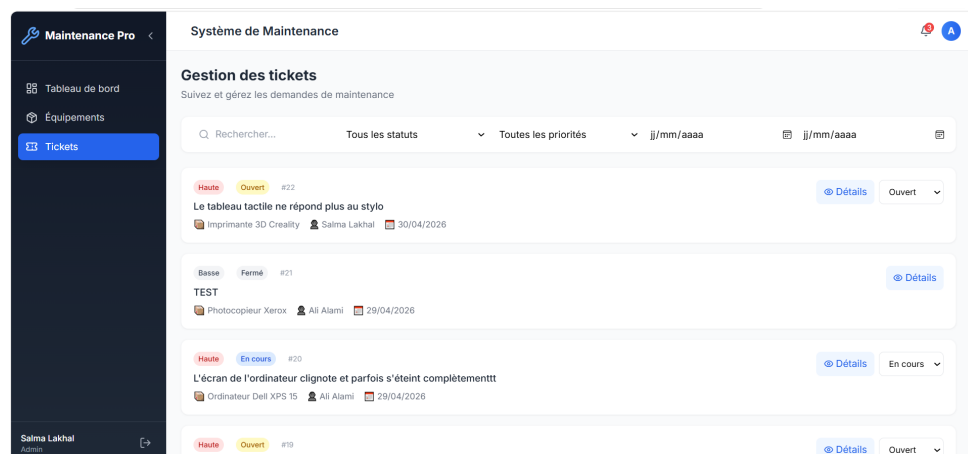


Figure 5.15 : Liste des tickets (Web)

La figure 5.16 montre le détail d'un ticket avec historique, commentaires et actions possibles (changement de statut, assignation).

Détail du ticket #5 [X]

Équipement Imprimante 3D Creality	Déclarant Ali Alami
Priorité Haute	Statut Fermé
Technicien assigné Bahia Jihan	Date de création 26/04/2026 02:38:31

Description

Le tableau tactile ne répond plus au stylo

Historique des actions

Ali Alami ticket_created : Ticket créé	26/04/2026 02:38:32
Salma Lakhal assign_technician : Bahia Jihan Ancienne valeur: non assigné	30/04/2026 03:07:19

Assigner un technicien

Figure 5.16 : Détail d'un ticket avec historique (Web)

5.4.5 Fonctionnalités principales implémentées

- **Authentification** : connexion sécurisée avec JWT.
- **Gestion des équipements** : CRUD complet (créer, lire, modifier, supprimer).
- **Gestion des tickets** : création, consultation, modification du statut.
- **Historique** : traçabilité complète via la table `ticket_logs`.
- **Tableau de bord** : statistiques (nombre de tickets, répartition par priorité, équipements les plus signalés).

5.5 Stratégie de tests

5.5.1 Tests unitaires

- **Backend** : tests manuels des endpoints avec Postman.
- **Mobile** : tests manuels sur émulateur (différentes tailles d'écran, versions Android).

5.5.2 Tests d'intégration

- Vérification de la communication entre l'application mobile et l'API.
- Vérification de la communication entre l'interface web et l'API.
- Validation du flux complet : création d'un ticket depuis mobile → consultation depuis web → changement de statut.

5.5.3 Tests système

- Test des différents rôles (user, tech, admin).
- Test de la persistance des données en base MySQL.
- Test de la sécurité (accès aux routes protégées, expiration du token JWT).

5.5.4 Outils de test

- **Postman** : tests des API REST.
- **Logcat Android** : débogage de l'application mobile.
- **Console navigateur (Inspecteur)** : débogage de l'interface web.

5.5.5 Résultats des tests

Voici un résumé des tests effectués :

Table 5.5 : Résultats des tests

Fonctionnalité testée	Résultat	Observations
Authentification (JWT)	Réussi	Token généré correctement
CRUD équipements	Réussi	Routes fonctionnelles
Création ticket (mobile)	Réussi	Ticket bien enregistré
Affichage liste tickets	Réussi	Filtrage par rôle OK
Historique des actions	Réussi	Logs bien affichés
Tableau de bord (statistiques)	Réussi	Graphiques cohérents
Sécurité (routes protégées)	Réussi	401 si token absent

5.6 Conclusion

Pour clore ce chapitre, nous dressons un bilan des décisions techniques et de réalisation :

- **Synthèse des choix techniques** : Les technologies sélectionnées (Node.js, React, Kotlin, MySQL) se sont avérées parfaitement adaptées aux besoins du projet. Elles ont permis un développement rapide, une bonne maintenabilité et le respect de la contrainte pédagogique (4 tables).
- **Retour sur l'architecture de déploiement et l'implémentation** : L'architecture orientée services a facilité le développement parallèle des trois composants (backend, mobile, web). La séparation en couches garantit la clarté du code.
- **Difficultés rencontrées** :
 - Gestion du token JWT dans l'application mobile (intercepteur Retrofit).
 - Rafraîchissement automatique de la liste des tickets après création.
 - Mise en place de l'historique des actions (ticket_logs).

Ces difficultés ont été résolues progressivement grâce à une documentation abondante et des tests itératifs.

- **Perspectives d'amélioration :**

- Ajout de notifications push (Firebase Cloud Messaging).
- Ajout d'une fonctionnalité de géolocalisation pour les tickets.
- Déploiement en production avec conteneurisation (Docker).
- Ajout de tests automatisés (JUnit pour Android, Jest pour React).

- **Transition vers le chapitre suivant :** Les éléments techniques présentés ici sont pleinement opérationnels. Le chapitre suivant présentera une démonstration concrète du système ainsi que les perspectives d'évolution.

Ce chapitre constitue ainsi un socle technique solide, garantissant que les choix de développement sont cohérents avec les objectifs du projet et prêts à être mis en œuvre lors de la phase de réalisation.

Conclusion générale

La conclusion générale met en lumière les principaux enseignements et aboutissements du projet. Elle récapitule les objectifs initiaux, synthétise les contributions et résultats obtenus, identifie les limites du travail réalisé, et propose des pistes pour des développements futurs.

▪ Récapitulation des objectifs initiaux :

- Ce projet, réalisé dans le cadre du module **Développement Web et Mobile 2** à l'ENS Marrakech, visait à concevoir et développer une plateforme complète de gestion de maintenance des équipements dans les salles de cours et laboratoires. L'objectif principal était de digitaliser le processus de signalement et de suivi des pannes, en remplaçant les méthodes traditionnelles (papier, emails) par une solution centralisée et efficace.
- Les objectifs spécifiques comprenaient : le développement d'une application mobile Android pour les utilisateurs finals, la création d'une API REST sécurisée (Node.js), la réalisation d'une interface web d'administration (React) pour les techniciens et administrateurs, et le respect de la contrainte pédagogique de quatre tables dans la base de données.
- Une évaluation des résultats montre que **tous les objectifs techniques et fonctionnels ont été atteints**. La plateforme permet désormais de créer, consulter, modifier et suivre des tickets de maintenance de manière fluide, avec une traçabilité complète via l'historique des actions.

▪ Contributions et résultats obtenus :

- **Backend** : Une API REST sécurisée (JWT, bcrypt) a été développée avec Node.js et Express, offrant des endpoints complets pour la gestion des utilisateurs, des équipements et des tickets. La base de données MySQL respecte la contrainte des quatre tables.
- **Application mobile Android** : Développée en Kotlin, elle permet aux utilisateurs de se connecter, de consulter leurs tickets, d'en créer de nouveaux (équipement, description, priorité) et de visualiser l'historique des actions.
- **Interface web** : Réalisée avec React et Tailwind CSS, elle offre un tableau de bord interactif avec des graphiques (Chart.js), une gestion complète des équipements (CRUD) et des tickets (filtres, changement de statut, assignation de technicien).
- **Sécurité** : L'authentification JWT garantit un accès sécurisé aux ressources, avec une gestion fine des rôles (utilisateur, technicien, administrateur). Les mots de passe sont hashés avec bcrypt.
- **Traçabilité** : La table `ticket_logs` enregistre l'intégralité des actions effectuées sur chaque ticket (création, changement de statut, assignation, commentaires), assurant une transparence totale.

▪ Limites du travail :

- **Notifications en temps réel** : La plateforme ne dispose pas encore d'un système de notifications push (Firebase Cloud Messaging) pour informer instantanément les techniciens lorsqu'un nouveau ticket est créé.

- **Géolocalisation** : L'application mobile ne propose pas de fonctionnalité de géolocalisation pour localiser précisément un équipement en panne.
- **Tests automatisés** : Les tests unitaires et d'intégration ont été effectués manuellement. Une automatisation (JUnit pour Android, Jest pour React) serait bénéfique pour la fiabilité à long terme.
- **Déploiement en production** : Actuellement, la plateforme tourne en environnement local. Un déploiement sur le cloud (AWS, Heroku) nécessiterait des configurations supplémentaires (variables d'environnement, gestion des secrets, base de données distante).

▪ **Perspectives et améliorations futures :**

- **Notifications push** : Intégrer Firebase Cloud Messaging (FCM) pour informer les techniciens en temps réel des nouvelles demandes.
- **Géolocalisation** : Ajouter la localisation GPS dans l'application mobile pour permettre de situer précisément l'équipement défaillant.
- **Amélioration de l'interface** : Enrichir le tableau de bord avec d'autres indicateurs (temps moyen de résolution, taux de satisfaction, etc.).
- **Tests automatisés** : Mettre en place des tests unitaires (JUnit) et d'intégration (Postman/Newman) pour garantir la robustesse du système.
- **Déploiement cloud** : Conteneuriser l'application avec Docker et la déployer sur une plateforme cloud (AWS, Heroku, ou Render) pour la rendre accessible en ligne.
- **Application iOS** : Étendre la solution à l'écosystème Apple en développant une version iOS (Swift) ou en utilisant des technologies multiplateformes (Flutter, React Native).
- **Export de rapports** : Ajouter une fonctionnalité d'export des tickets et statistiques au format PDF ou Excel.
- **Mot de passe oublié** : Implémenter complètement la fonctionnalité de réinitialisation par email (envoi de lien, token sécurisé).

Bilan général :

Ce projet a permis de développer une plateforme complète, fonctionnelle et sécurisée pour la gestion de maintenance des équipements. Il a couvert l'ensemble des aspects du développement web et mobile : conception, modélisation UML, développement backend (API REST), développement frontend (React), développement mobile (Kotlin), gestion de base de données, sécurité (JWT, bcrypt) et déploiement.

Acquis et compétences développées :

Ce travail a permis d'acquérir et de renforcer des compétences clés :

- Maîtrise du développement d'API REST avec Node.js et Express.
- Maîtrise du développement d'applications Android avec Kotlin (Retrofit, Coroutines, Material Design).
- Maîtrise du développement d'interfaces web modernes avec React et Tailwind CSS.
- Conception et gestion de bases de données relationnelles (MySQL).
- Mise en place d'une authentification sécurisée (JWT) et de la gestion des rôles.
- Respect des contraintes pédagogiques (4 tables) et des bonnes pratiques de développement.

Mot de la fin :

Ce projet constitue une base solide pour une plateforme de gestion de maintenance exploitable en contexte réel. Les perspectives d'amélioration identifiées ouvrent la voie à une évolution continue du système, répondant ainsi aux besoins croissants des établissements d'enseignement en matière de digitalisation des processus de maintenance.

Apport du projet

Cette section offre l'occasion de prendre du recul sur l'expérience acquise au cours de ce projet. Plutôt que de simplement lister les compétences techniques (maîtrise de divers langages de programmation, rédaction de rapports en LATEX, utilisation d'outils techniques divers), il s'agit ici de réfléchir sur l'ensemble du processus de recherche et de développement, sur la démarche de résolution de problèmes et sur l'impact de cette expérience sur votre parcours professionnel et personnel.

Retour sur les objectifs initiaux :

Au début du projet, les objectifs étaient clairement définis et orientés vers la résolution d'une problématique spécifique : la digitalisation du processus de maintenance des équipements dans les salles de cours et laboratoires. Nous visions à développer une plateforme complète composée de trois parties distinctes : une application mobile Android pour les utilisateurs, une API REST backend sécurisée, et une interface web d'administration pour les techniciens et administrateurs. Au fil de la réalisation, il est apparu que la démarche de recherche et d'expérimentation permettait non seulement de valider des choix techniques, mais également de développer une approche structurée de la résolution de problèmes. Les différentes étapes, de l'identification des besoins à la mise en œuvre des solutions (analyse des besoins, conception UML, développement, tests), ont constitué un véritable apprentissage en gestion de projet et en innovation.

Connaissances et compétences développées :

Parmi les connaissances et compétences développées au cours de ce projet, on peut citer :

- **Développement backend** : Maîtrise de **Node.js** et du framework **Express** pour la création d'API REST. Implémentation d'un système d'authentification sécurisé avec **JWT** (JSON Web Token) et hachage des mots de passe avec **bcrypt**. Gestion des rôles (utilisateur, technicien, administrateur) et des middlewares.
- **Développement mobile Android** : Acquisition d'une solide expérience en **Kotlin**, utilisation de **Retrofit** pour les appels HTTP, programmation asynchrone avec **Coroutines**, architecture **MVVM** (ViewModel, LiveData), et conception d'interfaces modernes avec **Material Design** (RecyclerView, CardView).
- **Développement web** : Maîtrise de **React** (composants, hooks, contexte) et de **Tailwind CSS** pour une interface utilisateur moderne et responsive. Création de graphiques interactifs avec **Chart.js**.
- **Base de données** : Conception et modélisation d'une base de données relationnelle **MySQL** respectant la contrainte pédagogique de quatre tables. Écriture de requêtes SQL préparées pour prévenir les injections SQL.
- **Outils de développement** : Utilisation avancée d'**Android Studio**, de **Visual Studio Code**, de **Git** pour le contrôle de version (dépôt GitHub), et de **Postman** pour les tests d'API.
- **Rédaction scientifique** : Rédaction du rapport en **LATEX**, ce qui a permis d'acquérir une rigueur dans la mise en forme et la structuration des documents techniques (gestion des sections, figures, tableaux, citations).

- **Gestion de projet** : Adoption d'une méthodologie **agile (Scrum)** avec un découpage en sprints, une planification des tâches, des revues régulières avec l'encadrant, et une adaptation continue face aux imprévus.

Défis rencontrés et leçons apprises :

Toutefois, ce projet a également présenté son lot de défis :

- **Intégration mobile-api** : La gestion du token JWT dans l'application mobile a nécessité la mise en place d'un intercepteur Retrofit (`AuthInterceptor`) pour ajouter automatiquement le token dans l'en-tête `Authorization`. Cette étape a été délicate mais formatrice.
- **Rafraîchissement de la liste des tickets** : Après création d'un ticket, la liste ne se mettait pas à jour automatiquement. La solution a été d'ajouter la méthode `onResume()` dans le fragment pour recharger les données à chaque retour.
- **Historique des actions** : La mise en place d'une table `ticket_logs` pour tracer toutes les actions a nécessité une réflexion approfondie sur la structure des logs et les déclencheurs d'écriture.
- **Respect de la contrainte des 4 tables** : Toute la conception a dû être pensée pour respecter cette contrainte pédagogique, ce qui a imposé une optimisation du schéma relationnel (regroupement des données dans les tables existantes).
- **Tests et débogage** : Les tests manuels sur plusieurs émulateurs Android et navigateurs web ont pris du temps, mais ils ont permis d'identifier et de corriger de nombreux bugs (gestion des erreurs, affichage des messages, validation des formulaires).

Ces obstacles ont été autant d'occasions de repenser la stratégie initiale et d'adapter les choix techniques en fonction des réalités du terrain.

Pistes d'amélioration pour une future réalisation :

Si ce projet devait être réalisé à nouveau, plusieurs axes pourraient être améliorés :

- **Planification plus approfondie** : Une phase d'analyse des risques plus rigoureuse aurait permis d'anticiper certaines difficultés techniques (notamment l'intégration mobile-api).
- **Tests automatisés** : Mettre en place des tests unitaires (JUnit pour Android, Jest pour React) et d'intégration (Postman/Newman) dès le début du projet pour gagner en fiabilité.
- **Documentation interne** : Une meilleure documentation du code (commentaires, README) dès le départ aurait facilité la maintenance et l'évolution du système.
- **Collaboration renforcée** : Travailler en binôme sur certaines parties complexes (notamment l'API et l'application mobile) aurait permis un partage de compétences et une résolution plus rapide des problèmes.
- **Déploiement continu** : Mettre en place un pipeline CI/CD (GitHub Actions, GitLab CI) pour automatiser les builds, les tests et le déploiement.

Impact sur le parcours professionnel :

Ce projet de fin d'études a constitué une expérience extrêmement enrichissante. Il m'a permis de :

- Mettre en pratique les connaissances théoriques acquises durant ma formation (développement web, mobile, bases de données, sécurité).
- Développer une vision globale d'un projet informatique complet, de l'analyse des besoins à la livraison d'une solution fonctionnelle.
- Renforcer mes compétences en résolution de problèmes et en autonomie.
- Préparer mon insertion professionnelle dans le domaine du développement full-stack et du génie logiciel.

Conclusion personnelle :

Ce projet a été une expérience formatrice tant sur le plan technique qu'humain. Il m'a appris à gérer un projet de A à Z, à faire des choix architecturaux cohérents, à m'adapter face aux difficultés et à livrer une solution fonctionnelle dans des délais contraints. Je suis fier du résultat obtenu : une plateforme de gestion de maintenance complète, sécurisée et ergonomique, qui répond aux besoins exprimés par l'encadrant. Cette expérience constitue un atout majeur pour la suite de mon parcours professionnel.

Bibliographie

- [1] Node.js Foundation. *Node.js Documentation*. <https://nodejs.org/en/docs/>, consulté le 20/04/2026.
- [2] Express.js. *Express - Guide de référence*. <https://expressjs.com/>, consulté le 20/04/2026.
- [3] Meta Platforms, Inc. *React - Documentation officielle*. <https://fr.legacy.reactjs.org/>, consulté le 20/04/2026.
- [4] Tailwind Labs. *Tailwind CSS - Documentation*. <https://tailwindcss.com/docs>, consulté le 20/04/2026.
- [5] Google Developers. *Android Developers Documentation*. <https://developer.android.com/docs>, consulté le 20/04/2026.
- [6] JetBrains. *Kotlin Programming Language*. <https://kotlinlang.org/docs/>, consulté le 20/04/2026.
- [7] Square, Inc. *Retrofit - Documentation*. <https://square.github.io/retrofit/>, consulté le 20/04/2026.
- [8] JSON Web Token (JWT). *JWT - Introduction*. <https://jwt.io/introduction>, consulté le 20/04/2026.
- [9] Niels Provos, David Mazières. *bcrypt - A Future-Adaptable Password Hashing Algorithm*. USENIX Security Symposium, 1999.
- [10] Oracle Corporation. *MySQL Documentation*. <https://dev.mysql.com/doc/>, consulté le 20/04/2026.
- [11] Chart.js contributors. *Chart.js - Documentation*. <https://www.chartjs.org/docs/latest/>, consulté le 20/04/2026.
- [12] GLPI Project. *GLPI - Gestionnaire Libre de Parc Informatique*. <https://glpi-project.org/>, consulté le 20/04/2026.
- [13] Object Management Group (OMG). *UML Specification*. <https://www.uml.org/>, consulté le 20/04/2026.
- [14] Ken Schwaber, Jeff Sutherland. *The Scrum Guide*. Scrum.org, 2020.
- [15] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Thèse de doctorat, Université de Californie, Irvine, 2000.

Sitographie

- **Node.js** : <https://nodejs.org/>
- **Express.js** : <https://expressjs.com/>
- **React** : <https://reactjs.org/>
- **Tailwind CSS** : <https://tailwindcss.com/>
- **Android Developers** : <https://developer.android.com/>
- **Kotlin** : <https://kotlinlang.org/>
- **Retrofit** : <https://square.github.io/retrofit/>
- **JWT** : <https://jwt.io/>
- **MySQL** : <https://dev.mysql.com/>
- **Chart.js** : <https://www.chartjs.org/>
- **GLPI** : <https://glpi-project.org/>
- **UML** : <https://www.uml.org/>

Annexe A

Annexes (Facultatif)

Les annexes regroupent des documents complémentaires — tels que des captures d'écran détaillées, des extraits de code significatifs, des schémas d'architecture complets ou des résultats de tests — qui, bien qu'importants pour une compréhension approfondie du projet, ne constituent pas l'essence même du rapport. Une annexe est destinée aux lecteurs souhaitant obtenir des informations supplémentaires ou une explication plus complète sur certains aspects du projet. Il est recommandé d'utiliser une annexe par idée ou par thème afin de maintenir une organisation claire et structurée. L'inclusion d'annexes est facultative. Si vous estimez que les informations supplémentaires ne sont pas indispensables à la compréhension du projet, il est préférable de ne pas les ajouter. En effet, inclure des documents non pertinents ou superflus pourrait augmenter indûment le nombre total de pages du rapport et distraire le lecteur.

Annexe A : Extraits de code significatifs

a.1 Intercepteur JWT (Android)

```
class AuthInterceptor(private val tokenManager: TokenManager) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val request = chain.request()
        val token = tokenManager.getToken()
        val newRequest = if (token != null) {
            request.newBuilder()
                .header("Authorization", "Bearer $token")
                .build()
        } else {
            request
        }
        return chain.proceed(newRequest)
    }
}
```

a.2 Middleware d'authentification (Node.js)

```
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];
    if (!token) return res.status(401).json({ error: 'Token manquant' });
    jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
        if (err) return res.status(403).json({ error: 'Token invalide' });
        req.user = user;
    });
    next();
}
```

```

        next();
    });
};

```

a.3 Configuration de Retrofit (Android)

```

object RetrofitClient {
    private const val BASE_URL = "http://10.0.2.2:3000/api/"
    val apiService: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .client(OkHttpClient.Builder()
                .addInterceptor(AuthInterceptor(TokenManager(context)))
                .build())
            .build()
            .create(ApiService::class.java)
    }
}

```

Annexe B : Script SQL de création des tables

```

-- Table des utilisateurs
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('user', 'tech', 'admin') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des équipements
CREATE TABLE equipments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    room VARCHAR(50) NOT NULL,
    type VARCHAR(50) NOT NULL,
    status ENUM('operational', 'out_of_service') DEFAULT 'operational',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des tickets
CREATE TABLE maintenance_tickets (
    id INT PRIMARY KEY AUTO_INCREMENT,
    equipment_id INT NOT NULL,
    user_id INT NOT NULL,
    technician_id INT NULL,
    description TEXT NOT NULL,
    priority ENUM('low', 'medium', 'high') DEFAULT 'medium',
    status ENUM('open', 'in_progress', 'resolved', 'closed') DEFAULT 'open',

```

```

    photo_url VARCHAR(500) NULL,
    resolution_comment TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (equipment_id) REFERENCES equipments(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (technician_id) REFERENCES users(id) ON DELETE SET NULL
);

-- Table des logs
CREATE TABLE ticket_logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    ticket_id INT NOT NULL,
    user_id INT NOT NULL,
    action VARCHAR(100) NOT NULL,
    old_value TEXT NULL,
    new_value TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ticket_id) REFERENCES maintenance_tickets(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

Annexe C : Résultats des tests

Table A.1 : Résultats des tests fonctionnels

Fonctionnalité testée	Résultat	Observations
Authentification (JWT)	Succès	Token généré correctement
CRUD équipements	Succès	Routes fonctionnelles
Création ticket (mobile)	Succès	Ticket bien enregistré
Affichage liste tickets	Succès	Filtrage par rôle OK
Historique des actions	Succès	Logs bien affichés
Tableau de bord web	Succès	Graphiques cohérents
Sécurité (routes protégées)	Succès	401 si token absent

Annexe D : Guide d'installation rapide

d.1 Backend (Node.js)

```

cd backend
npm install
npm run dev

```

d.2 Interface web (React)

```

cd web-admin
npm install
npm start

```

d.3 Application mobile (Android)

1. Ouvrir le projet dans Android Studio

2. Modifier Constants.BASE_URL si nécessaire
3. Build → Clean Project
4. Run (émulateur ou appareil physique)

d.4 Base de données (MySQL)

1. Créer une base de données "projet_maintenance"
2. Exécuter le script SQL (Annexe C)
3. Configurer .env (backend)

Salma LAKHAL

Maintenance des équipements (salles / laboratoires)

Le présent projet vise à développer une plateforme complète de gestion de maintenance des équipements dans les salles de cours et laboratoires. L'objectif principal est de digitaliser le processus de signalement et de suivi des pannes, en remplaçant les méthodes traditionnelles (papier, courriels) par une solution centralisée, efficace et sécurisée.

Pour atteindre cet objectif, une approche méthodologique rigoureuse a été adoptée, combinant une analyse fonctionnelle détaillée, une conception modulaire (UML) et l'implémentation de technologies modernes. La plateforme se compose de trois parties distinctes : une application mobile Android (Kotlin, Retrofit, Coroutines) pour les utilisateurs finaux, une API REST sécurisée (Node.js, Express, JWT, bcrypt) pour la logique métier, et une interface web d'administration (React, Tailwind CSS, Chart.js) pour les techniciens et administrateurs. La base de données MySQL respecte la contrainte pédagogique de quatre tables (utilisateurs, équipements, tickets, logs).

Les contributions majeures du projet incluent la mise en œuvre d'une interface utilisateur intuitive (mobile et web), l'intégration d'un système performant de gestion des tickets (création, consultation, changement de statut, assignation de technicien), une traçabilité complète via l'historique des actions (table `ticket_logs`), et la validation de la solution via des tests exhaustifs (authentification JWT, CRUD complet, protection des routes). Ces résultats traduisent une amélioration significative en termes de réactivité, de traçabilité et de sécurité par rapport aux solutions existantes (GLPI, OTRS, etc.).

Ce projet a permis d'acquérir des compétences avancées en développement mobile (Kotlin, Android), développement backend (Node.js, API REST, JWT), développement frontend (React, Tailwind CSS, graphiques), conception de bases de données (MySQL, modélisation UML), et gestion de projet (méthodologie Scrum, rédaction scientifique en $\text{\LaTeX}$). En outre, il ouvre des perspectives prometteuses pour de futurs développements, tels que l'ajout de notifications push (Firebase Cloud Messaging), la géolocalisation des tickets, l'export de rapports (PDF/Excel), le déploiement cloud (Docker, AWS) et la mise en place de tests automatisés (CI/CD).

L'ensemble de ces réalisations témoigne de l'engagement à proposer une solution à la fois innovante, ergonomique et efficace, répondant aux problématiques identifiées dès le début du projet.

Mots clés : gestion de maintenance, application mobile Android, API REST, JWT, React, Node.js, Kotlin, tickets, historique des actions, traçabilité